本科毕业论文（设计）

论文（设计）题目

论文（设计）英文题目

学　　院：_____

专　　业：_____

学生姓名：_____

学　　号：_____

指导教师：_____

北京交通大学

2026 年 4 月

学士论文版权使用授权书

本学士论文作者完全了解北京交通大学有关保留、使用学士论文的规定。特授权北京交通大学可以将学士论文的全部或部分内容编入有关数据库进行检索，提供阅览服务，并采用影印、缩印或扫描等复制手段保存、汇编以供查阅和借阅。

（保密的学位论文在解密后适用本授权说明）

学位论文作者签名： 指导教师签名：

签字日期： 年 月 日 签字日期： 年 月 日

学士论文诚信声明

本人声明所呈交的毕业论文（设计），题目_____ 是本人在指导教师的指导下，独立进行研究工作所取得的成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京交通大学或其他教育机构的学位或证书而使用过的材料。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

本人签名：_____ 日期：_____

中文摘要

摘要：［鼠标左键单击选择该段落，输入替换之。内容为小四号宋体。］中文摘要应将论文的内容要点简短明了地表达出来，约 400 字左右，字体为宋体小四号。内容应包括工作目的、研究方法、成果和结论。要突出本论文的创新点，语言力求精炼。为了便于文献检索，应在本页下方另起一行注明论文的关键词（3-5 个），如有可能，尽量采用《汉语主题词表》等词表提供的规范词。图 X 幅，表 X 个，参考文献 X 篇。

关键词：［请输入关键词（3-5），以分号分隔。］

ABSTRACT

ABSTRACT:"[鼠标左键单击选择该段落,输入替换之。内容为小四号 Times New Roman。]"与中文摘要内容要相对应。

KEYWORDS：[请输入英文关键词，与中文关键词保持一致。以分号分隔。]

目　　录

中文摘要 I
ABSTRACT II
目　　录 III
1　引言 5
1.1　研究背景与研究意义 5
1.2　国内外研究现状 5
1.3　协同计算研究现状 6
1.4　现有研究工作不足 6
1.5　论文主要工作 6
1.6　本文组织结构 6
2　关键理论与技术 7
2.1　奇异值分解与矩阵最优低秩近似 7
2.2　基于 TRANSFORMER 架构的深度学习模型 8
2.3　KV 缓存 10
2.4　算子 11
2.5　计算图 11
2.6　推理框架 12
2.7　4-BIT 权重量化 12
2.8【2 级标题，小三号黑体字】 ..... 错误!未定义书签。
2.8.1【3 级标题，四号黑体字】 ..... 错误!未定义书签。
3　基于 SVD 的多核协同卸载计算方法建模 14
3.1　概述 14
3.2　基于 SVD 的卷积矩阵化访存优化 14
3.3　基于 SVD 的权重矩阵计算任务卸载方法 15
3.4　时限约束下的弹性融合问题建模 17
3.5　不同模型类型下的协同方向选择策略 18
4　面向无原始数据场景的协同策略精度评估算法 20
4.1　概述 20
4.2　问题定义与符号约定 20
4.3　损失函数设计 20
4.4　基于梯度下降的输入空间拟合算法 21
4.4.1　面向 Transformer 架构模型的输入空间拟合方法 21
4.4.2　面向卷积架构模型的输入空间拟合方法 21
5　基于负载感知的边端协同联合实时调度策略 22

5.1 基于动态规划的计算任务分配策略 22
5.2 面对高负载场景的边端协同推理 28
6 结论【1 级标题，三号黑体字】 31
参考文献 42
致 谢 43

目录说明：

1 级标题 （小四、黑体、左对齐、空 1 字符）

2 级标题 （小四、宋体、首行缩进 1 字符）

3 级标题 （小四、宋体、首行缩进 2 字符）

1 引言

1.1 研究背景与研究意义

随着个人智能助手与 Agent 形态向 “长期驻留、持续感知、多步决策” 的方向演进，推理链路对交互时延稳定性、数据隐私可控性提出更高要求。与此同时，产业界正推动小参数大模型向端侧与个人侧设备迁移[1]。

本研究通过 “本地推理 + 可信协同” 的方式兼顾交互实时性与隐私约束，构建以个人电脑为主要承载、移动端可弹性参与的通用协同推理机制，不仅能够提高个人侧算力的整体利用效率，还能够在网络波动与设备负载变化时保持可预测的性能退化与可控的精度风险，为个人智能助手的私有化部署提供可持续的工程支撑。

应对个人智能助手与 Agent 私有化的趋势：以个人智能助手为代表的新一代应用形态正在快速普及，其核心特征是持续接入多模态上下文（语音、图像、地理位置、日程与应用状态），并以 “多步任务编排” 的方式完成搜索、填写、操作与决策等复杂流程。在该类场景中，交互链路对端到端时延与稳定性高度敏感，同时用户数据具有强隐私属性，使 “尽可能在本地完成推理与决策” 成为重要诉求。

云端推理会增加隐私泄露风险：个人电脑端智能助手会持续接触本地敏感信息与细粒度上下文（文档内容、应用状态、输入片段、剪贴板、日程与通信记录等）。当核心推理依赖云端时，这些数据需要跨网络传输并在远端处理，使隐私保护不再只由本地设备决定，而同时受传输链路、云端环境与第三方组件影响，暴露面扩大。并且云端常见的日志留存、故障排查采样与多租户共享机制，进一步提高数据被留存或越权访问的风险；一旦发生配置或权限问题，泄露范围与追溯成本将显著上升。

小参数大模型云侧部署造成资源浪费：端侧与边侧计算能力的持续提升（手机 SoC 与个人电脑 CPU/GPU）已使小参数大模型具备本地可交互推理的现实基础：现有主流推理框架中，7B 量级模型经过量化后在端侧可达到可用的生成速度[13]并且具有足够的端侧任务能力[14,15,16]。在此背景下，若仍将该类模型作为云端常态推理服务，会在 “本地即可满足” 的请求上持续占用云侧 GPU/CPU 预算与并发容量，形成不必要的算力消耗与资源错配。更合理的体系是将个人电脑与手机共同纳入协同计算：当个人电脑侧负载升高或前台交互受资源竞争影响时，由手机侧分担可拆分计算以降低边侧 CPU/GPU 持续高占用带来的系统抖动；当个人电脑侧空闲时维持其主执行以减少跨设备协同开销，从而在个人侧形成可弹性调度的私有算力池，并将云侧资源用于端侧能力边界之外的补充推理。

1.2 国内外研究现状

为应对这些挑战，已有工作从多个方向展开探索。模型压缩技术，例如剪除冗余参数，或采用 Once-for-All（OFA）训练方法 [1]，旨在生成适合边缘部署的紧凑模型。然而，这些方法通常需要完整访问原始训练数据集以执行剪枝，或者依赖于大规模超网络训练，因此部署过程缓慢且资源开销较大。类似地，知识蒸馏通过利用大型教师模型训练一个轻量级学生模型，但同样需要大规模数据集，这限制了其在快速部署场景中的

实用性。

另一类工作聚焦于云边协同推理，即将一个神经网络划分为两个部分：一部分运行在边缘侧，另一部分运行在云端。例如，Neurosurgeon [2] 利用移动端 SoC 将模型分割到不同设备之间。然而，它通过预训练回归模型来确定划分位置，这限制了其对新架构和新平台的适应能力。AgileNN [3] 则借助可解释人工智能（XAI）将每个输入划分为重要部分和非重要部分，并分别分配给部署在云端和边缘端的高精度、低精度模型分支。该方法提升了效率，但严重依赖于对数据重要性的准确估计；并且每当原始模型更新时，都需要重新训练边缘侧模型，从而使频繁部署的成本高、耗时长。Auto-Split [4] 将模型划分表述为一个同时考虑本地计算能力、云端容量和网络带宽的优化问题，但其搜索空间过大，不适合快速部署。

在实际应用中，快速部署尤为重要。神经网络模型正快速演进 [5]，开发者需要尽快将更新传播到边缘设备。然而，当前部署流水线通常十分耗时：它们往往涉及模型重训练或剪枝、超参数调优，以及在不同部署策略下进行性能分析。每一轮更新周期都可能需要数小时甚至数天，这取决于模型复杂度和系统异构性。因此，实现从模型剪枝、划分到部署的快速端到端流程，对于实际边缘智能系统至关重要。

1.3 协同计算研究现状

1.4 现有研究工作不足

缺乏统一的协同计算策略：

依赖原始数据集的参与：

难以应对实际场景的硬件负载和网络波动变化：

1.5 论文主要工作

提出一种面向边端协同推理的弹性补偿卸载策略，该策略采用 xxx 核心思路，实现 xxx。

面向无原始数据场景，提出一种协同策略精度评估算法，该算法采用 xxx 的核心思路，实现 xxx。

应对实际场景的硬件负载和网络波动，提出一种多目标联合实时调度算法，该算法采用 xxx 的核心思想，实现 xxx

1.6 本文组织结构

2 关键理论与技术

2.1 奇异值分解与矩阵最优低秩近似

奇异值分解的数学表示形式：奇异值分解（Singular Value Decomposition，SVD）是一种重要的矩阵分解方法，在数值计算、信号处理、模式识别等领域具有广泛应用。其数学表示为：对于任意实矩阵

$$A \in \mathbb{R}^{m\times n} \tag{2-1-1}$$

总存在两个正交矩阵 $U \in \mathbb{R}^{m\times m}$、$V \in \mathbb{R}^{n\times n}$ 以及一个对角矩阵 $\Sigma \in \mathbb{R}^{m\times n}$，使得矩阵 A 可表示为

$$A = U\Sigma V^T \tag{2-1-2}$$

其中，Σ 的对角线元素称为矩阵 A 的奇异值，通常按非增顺序排列，即

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 \tag{2-1-3}$$

其中，$r = \mathrm{rank}(A)$，为矩阵的秩。

基于奇异值分解的矩阵近似：基于式 2-1-2 可将矩阵的奇异值分解进一步展开得到

$$A = \sum_{i=1}^{r} \sigma_i\, u_i v_i^T \tag{2-1-4}$$

其中，u_i 和 v_i 分别为矩阵 U 与矩阵 V 的第 i 个列向量。由此可见，矩阵 A 可以表示为若干个秩为 1 的矩阵之和，而奇异值 σ_i 则表征各分量对原矩阵整体结构的贡献程度。奇异值越大，说明对应分量对矩阵主要特征的影响越显著；奇异值越小，则说明该分量对整体结构的贡献相对有限。基于这一性质，若仅保留前 k 个较大的奇异值及其对应的奇异向量，则可构造矩阵 A 的低秩近似

$$A_k = \sum_{i=1}^{k} \sigma_i\, u_i v_i^T = U_k \Sigma_k V_k^T, \qquad k < r \tag{2-1-5}$$

由于矩阵 A_k 仅由前 k 个奇异分量构成，因此其秩不超过 k，从而形成了原矩阵的低秩表示。该近似的基本思想是：矩阵中的主要信息通常集中在前若干个较大的奇异值所对应的分量中，而后续较小奇异值所代表的成分更多体现为次要结构或局部细节。因此，通过舍弃较小奇异值对应的部分，可以在尽量保留原矩阵主要结构特征的同时，实现对矩阵复杂度的有效降低。

奇异值分解的最优低秩近似性质：奇异值分解能够用于低秩近似的理论依据，可由 Eckart–Young 定理[?]给出。该定理表明，在所有秩不超过 k 的矩阵中，由前 k 个奇异值及其对应奇异向量构成的矩阵 A_k，是原矩阵 A 在 Frobenius 范数或二范数意义

下的最优近似。即对于任意满足 $\mathrm{rank}(B) \le k$ 的矩阵 B，恒有

$$\| A - A_k \|_F \le \| A - B \|_F \tag{2-1-6}$$

进一步地，其误差可表示为

$$\| A - A_k \|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2 \tag{2-1-7}$$

这表明，低秩近似所带来的误差恰好由被舍弃的奇异值所决定。若后续奇异值较小，则说明原矩阵的大部分信息已集中于前 k 个奇异值分量中，此时利用 A_k 替代 A 所引起的信息损失较小。

2.2 基于 Transformer 架构的深度学习模型

基于 Transformer 架构的大语言模型是当前自然语言处理领域的主流技术路线。该类模型通常通过大规模语料预训练获得通用语言建模能力，并通过指令微调、对齐训练等方式进一步提升在问答、摘要、翻译、代码生成和对话等任务中的表现。其核心基础是自注意力机制，该机制能够建模序列中任意位置之间的依赖关系，从而克服传统循环神经网络在长距离建模上的局限。

Transformer 的基本结构：Transformer 架构的模型通常由自注意力模块、前馈神经网络模块、残差连接和层归一化等部分组成。其中，自注意力模块负责建模序列内部各位置之间的相关性，而前馈神经网络则对每个位置上的特征表示进行进一步的非线性映射与变换，从而增强模型的表示能力。

自注意力模块：自注意力机制可以通过如下数学公式表示：设输入序列的隐藏表示为

$$X \in \mathbb{R}^{n \times d} \tag{2-2-1}$$

其中，n 表示序列长度，d 表示隐藏层维度。首先通过三个不同的线性变换，将输入表示 X 分别映射为查询矩阵、键矩阵和值矩阵：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V \tag{2-2-2}$$

其中，W_Q、W_K 和 W_V 为可学习的参数矩阵。由此，输入序列中每个位置的表示都会被投影到查询、键和值三个不同的子空间中，以支持后续的相关性计算与信息聚合。在得到 Q、K 和 V 后，Transformer 利用缩放点积注意力计算输出：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2-2-3}$$

其中，d_k 表示键向量维度。矩阵 QK^T 用于衡量序列中各位置之间的相关程度，经过缩放和 softmax 归一化后得到注意力权重，再与矩阵 V 相乘，从而实现对上下文信息的加权聚合。该机制使模型能够根据当前输入位置与其他位置之间的关联强度，自适应地提取对当前表示最有价值的上下文信息。

前馈神经网络模块：在注意力计算之后，Transformer 通常还包含位置前馈神经网络（Feed-Forward Network，FFN）。FFN 一般由两个线性变换层及中间的非线性激活函

数构成，其典型形式可表示为

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x + b_1) + b_2 \qquad (2-2-4)$$

其中，W_1 和 W_2 为可学习权重矩阵，b_1 和 b_2 为偏置项，$\phi(\cdot)$ 为非线性激活函数，如 ReLU、GELU 或 SiLU。与自注意力模块主要负责不同位置之间的信息交互不同，FFN 的作用在于对每个位置上的隐藏表示进行独立而深层的非线性变换，从而增强模型的表达能力。

基于 Decoder-only 架构的自回归推理机制：从结构上看，Transformer 通常由多个相同的基本层堆叠而成，每一层内部依次包含注意力子层和前馈网络子层，并通过残差连接和层归一化提高训练稳定性与表达能力。对于当前主流的大语言模型而言，广泛采用的是 Decoder-only 结构。该结构以自回归方式生成文本，即在每一步根据已有上下文预测下一个词元。其推理过程通常可分为 Prefill 和 Decode 两个阶段。Prefill 阶段对输入提示进行整体编码，需要处理全部上下文，具有较强的并行性和较高的吞吐需求；Decode 阶段则逐词元迭代生成，单步计算规模较小，但对响应时延更为敏感。这种阶段性差异使得不同硬件后端在不同阶段可能呈现出不同的适配优势。

因果掩码机制：在自回归生成场景下，Transformer 还需要引入因果掩码（Causal Mask）机制，以保证模型在预测当前位置输出时只能利用当前位置及其之前的上下文信息，而不能访问未来位置的内容。该约束是 Decoder-only 结构能够实现自回归生成的基础。若不加限制，注意力计算中的每个位置都可以与序列中所有位置进行交互，这将导致模型在训练或推理时提前获取后续 token 的信息，从而破坏因果生成过程。

设注意力得分矩阵为

$$S = \frac{QK^T}{\sqrt{d_k}} \qquad (2-2-5)$$

其中，矩阵 $S \in \mathbb{R}^{n\times n}$ 的第 i 行第 j 列元素表示序列中第 i 个位置对第 j 个位置的相关性评分。在标准自注意力中，所有位置之间均可参与计算；而在自回归场景中，需要引入掩码矩阵 M，使得当 $j > i$ 时，对应位置的得分被屏蔽。结合式 2-2-3，带因果掩码的注意力可表示为

$$Attention(Q,K,V) = softmax\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V \qquad (2-2-6)$$

其中，掩码矩阵 M 通常定义为

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \qquad (2-2-7)$$

由于 softmax 运算会将极小值对应的位置权重压缩至接近 0，因此未来位置的信息不会参与当前位置的表示计算，从而使得模型在第 i 个位置上只能关注前 i 个位置，从而满足自回归生成所要求的单向依赖关系。

模型参数分布：从模型结构与计算开销两方面来看，Transformer 中不同模块的重要性具有明显差异。首先，在参数分布上，前馈神经网络（FFN）通常是模型中最主要的参数承载模块之一。相较于注意力模块中用于生成查询、键和值的线性映射，FFN 一般包含两层线性变换，其中中间隐层维度通常会相对于隐藏维度进行较大比例的扩展，

再映射回原始维度，因此在许多 Transformer 模型中，FFN 往往占据更高的参数比例。

计算复杂度分析：在计算量分布上，FFN 与注意力模块的主导地位会随序列长度变化而变化。对于长度为 n、隐藏维度为 d 的输入序列，FFN 对每个位置独立执行相同的线性变换，其总体计算复杂度通常与 n 成线性关系，可近似表示为 $O(nd^2)$；而自注意力模块除线性投影外，还需要计算序列中各位置之间的两两相关性，其核心计算复杂度通常为 $O(n^2d)$。因此，当序列长度 n 较小时，FFN 往往是主要计算来源；而当序列长度 n 较大时，注意力模块相关开销会随 n^2 快速增长，从而逐渐成为整体计算瓶颈。

2.3 KV 缓存

KV 缓存（Key-Value Cache）是 Transformer 自回归推理过程中常用的一种加速机制，其核心思想是在生成过程中保存历史 token 对应的键矩阵 K 和值矩阵 V，从而避免在后续解码步骤中对已处理上下文重复计算。该机制直接来源于 Transformer 自注意力计算的数学形式，是提升大语言模型解码效率的重要基础。

对于自回归生成任务，由式 2-2-1 和式 2-2-2 可知，模型在第 t 个时间步生成新 token 时，其输入序列长度由 $t-1$ 增加到 t。若不使用缓存机制，则在每一步解码时都需要对整个当前序列重新计算

$$Q_t = X_t W_Q, \qquad K_t = X_t W_K, \qquad V_t = X_t W_V,$$

其中，X_t 表示长度为 t 的当前输入表示。这导致在第 t 步计算注意力时，历史 token 对应的键和值会被重复计算，而这些历史 token 的表示在因果掩码约束下对过去部分已固定，不会因为新 token 的加入而改变。也就是说，在解码阶段，前 $t-1$ 个位置对应的 K 和 V 在第 t 步与第 $t-1$ 步相比是相同的，因此没有必要重复生成。

基于这一特点，可将第 t 步输入拆分为历史部分和当前新增 token 部分。设新增 token 对应的隐藏表示为 x_t，则只需计算当前时刻的新查询、键和值：

$$q_t = x_t W_Q, \qquad k_t = x_t W_K, \qquad v_t = x_t W_V.$$

与此同时，将历史步骤已得到的键和值分别记为

$$K_{1:t-1} = [k_1, k_2, \dots, k_{t-1}]^T, \qquad V_{1:t-1} = [v_1, v_2, \dots, v_{t-1}]^T.$$

则第 t 步只需将新生成的 k_t 和 v_t 追加到缓存中，形成

$$K_{1:t} = [K_{1:t-1}; k_t], \qquad V_{1:t} = [V_{1:t-1}; v_t].$$

此时，当前 token 的注意力计算可写为

$$\mathrm{Attention}(q_t, K_{1:t}, V_{1:t}) = \mathrm{softmax}\left(\frac{q_t K_{1:t}^T}{\sqrt{d_k}}\right) V_{1:t}.$$

由此可见，在解码阶段，每一步只需要计算当前 token 对应的 q_t、k_t 和 v_t，并结合历史缓存的 K、V 完成注意力计算，而不必重新处理全部历史序列。

从计算复杂度角度看，若不使用 KV 缓存，则第 t 步需要重新计算长度为 t 的全部键和值表示，并对整个前缀重复执行相关矩阵运算；而引入 KV 缓存后，仅需计算新增 token 对应的一组 k_t 和 v_t，从而显著降低重复计算开销。尤其在长上下文和逐 token 解码场景下，缓存机制能够有效减少解码阶段的冗余计算，提高推理吞吐与响应效率。

2.4 算子

算子是深度学习模型执行过程中的基本计算单元，是构成计算图节点的核心元素。从形式上看，算子接收一个或多个输入张量，经过特定运算后生成输出张量。常见算子包括卷积、矩阵乘法、加法、归一化、激活函数、池化和拼接等。深度学习模型的推理过程，本质上是各类算子按照依赖关系有序执行的过程。

不同模型结构往往由不同类型的算子组合而成。例如，卷积神经网络主要依赖卷积、池化和全连接算子，而 Transformer 模型则更多依赖矩阵乘法、注意力、归一化和非线性激活算子。尽管模型层级结构复杂，但从推理系统实现角度看，最终都会被拆解为若干基础算子及其组合形式。

算子的性能直接决定模型整体推理效率。在端侧设备上，大量时间往往消耗在少数关键算子上，例如卷积层中的卷积算子和大语言模型中的矩阵乘法算子。因此，推理框架通常会针对这些高开销算子进行专门优化，包括并行化执行、向量化加速、内存对齐、算子融合以及调用专用硬件指令或加速库等。

此外，不同硬件后端对算子的支持能力并不相同。有些算子适合在 GPU 或 NPU 上高效执行，而有些控制类、数据重排类或轻量算子在 CPU 上执行更为灵活。某些硬件后端还要求输入张量遵循特定的数据布局或对齐方式，这进一步增加了算子映射与跨后端调度的复杂性。因此，在异构推理系统中，算子不仅是最基本的执行单位，也是后端划分与性能优化的关键对象。

本文研究中涉及的模型切分与协同执行，最终需要落实到具体算子的划分、调度与优化上。因此，对算子概念及其系统层意义进行说明，是后续分析的重要前提。

2.5 计算图

计算图是描述神经网络计算过程的一种图结构表示方式，用于刻画数据在各个运算节点之间的流动关系。通常情况下，计算图由节点和边组成，其中节点表示具体运算或变量，边表示张量数据之间的依赖关系。借助计算图，神经网络模型可以从“代码级过程描述”转换为“结构化执行描述”，从而便于优化、调度和跨平台部署。

在深度学习推理过程中，计算图一般表现为有向无环图。图中的输入节点表示模型输入或常量参数，中间节点表示卷积、矩阵乘法、归一化、激活函数等运算，输出节点表示最终结果。通过分析图中节点间的依赖关系，推理系统可以确定各算子的执行顺序，并进一步开展图级优化。

计算图的一个重要价值在于支持全局优化。由于模型整体被抽象为统一的图结构，推理框架可以在图层面进行算子融合、冗余节点消除、常量折叠、内存复用和跨设备划分。例如，当两个相邻算子可合并为一个复合算子时，可减少中间张量写回与读取开销；当某些分支在推理时无需重复计算时，可通过图优化降低整体执行代价。

在异构协同推理场景下，计算图还可以作为任务划分的重要依据。不同子图可根据其计算特征、数据依赖关系和硬件适配性，被映射到不同执行后端。例如，将高并行矩阵计算密集的部分映射到 NPU，将控制逻辑简单、时延敏感的部分映射到 CPU，均可视为对计算图进行结构化切分与调度。因此，计算图是实现跨设备协同推理的重要中间表示。

本文中涉及的模型切分、算子映射和阶段化执行，本质上都建立在计算图描述的基础之上。对计算图原理的理解，有助于分析深度学习模型在不同硬件平台上的执行机制与优化路径。

2.6 推理框架

推理框架是指用于支持已训练神经网络模型在目标设备上高效执行的软件系统。其主要功能包括模型加载、计算图构建、算子调度、内存管理、硬件适配和执行优化等。一般而言，推理框架处于模型表示与底层硬件之间的中间层。一方面，它负责将来自训练框架或模型导出工具的模型文件解析为内部可执行表示；另一方面，它需要面向 CPU、GPU、NPU 等不同硬件后端，完成算子映射、张量布局转换和执行调度。典型推理框架通常支持多种模型格式，并提供图优化、算子融合、常量折叠、量化执行和并行调度等优化机制，以提升整体推理性能。

【图 2-6-1：推理框架示意图】

在端侧和边缘设备中，推理框架的重要性更加突出。由于设备资源受限，模型执行不仅取决于神经网络本身，还受到内存容量、缓存结构、带宽限制、异构硬件能力和功耗预算等因素影响。因此，高性能推理框架往往需要针对具体平台进行深度适配。例如，在移动端系统中，推理框架需要综合考虑 CPU 与 NPU 的调用开销、数据搬移成本以及不同硬件后端对张量存储格式的要求。

对于云边协同场景，推理框架还承担着模型切分与跨节点协同执行的支撑作用。当模型的一部分在边端执行、另一部分在云端执行时，推理框架需要能够处理阶段间数据传输、计算衔接和状态保持等问题。若推理任务具有动态切换需求，还需要具备基于运行态进行调度决策的能力。因此，推理框架不仅是模型执行载体，也是连接算法优化与系统优化的重要桥梁。

2.7 4-Bit 权重量化

4-Bit 权重量化核心思想是将高精度浮点权重按固定大小划分为若干数据块，并在每个数据块内部用统一的缩放因子和 4 比特整数编码对原始权重进行近似表示。该方法通过牺牲部分数值精度，显著降低模型权重的存储开销和数据搬移成本，从而为低资源条件下的模型部署提供支持。根据其定义，Q4_0 通常以 32 个权重为一个量化块，每个量化块保存一个缩放因子以及对应的 4 比特量化结果。

设某一量化块中的原始浮点权重为

$$w_1, w_2, \dots, w_{32},$$

Q4_0 首先为该块选取一个统一的缩放因子 d，然后将块内每个权重映射为一个 4 比特

整数 q_i。在编码表示中，

$$q_i \in \{0,1,\dots,15\}.$$

虽然存储时使用的是无符号 4 比特整数，但其数值意义可视为以 8 为中心的对称量化。相应地，反量化后的近似权重可表示为

$$\hat{w}_i = d(q_i - 8).$$

由此可见，Q4_0 本质上采用的是“单一块级缩放因子 + 对称 4 比特离散码本”的表示方式，即在每个块内使用统一尺度将连续浮点权重压缩到有限个离散重建点上。

从算法流程上看，Q4_0 的量化过程可以概括为以下几个步骤。首先，将权重张量按顺序划分为长度为 32 的多个量化块；其次，在每个块内依据该块权重的分布情况确定统一的缩放因子 d；然后，将块内每个浮点权重按比例映射为对应的 4 比特整数编码 q_i；最后，在推理计算时通过

$$\hat{w}_i = d(q_i - 8)$$

将其恢复为近似浮点值。由于块内全部权重共享同一个缩放因子，Q4_0 的表示形式较为简单，量化存储与反量化计算过程都具有较低复杂度。

从压缩效果来看，Q4_0 将每个权重的表示位宽由高精度浮点数降低为 4 bit，并为每个量化块额外保存一个缩放因子，因此其平均存储成本远低于 FP16 或 FP32 表示。模型权重经量化后，整体参数体积显著减小，相应地，推理过程中的权重加载和内存带宽开销也会降低。这种收益在大规模模型中尤为明显，因为模型推理的性能瓶颈往往不仅来自计算本身，还受到权重访存效率的直接影响。

然而，Q4_0 也存在一定局限。由于一个量化块内的全部权重共享同一缩放因子，且每个权重只能映射到有限个离散值，因此量化后必然会引入近似误差。当块内权重分布较集中时，这种误差通常较小；但当同一块内同时存在幅值差异较大的权重时，统一的块级缩放会降低表示精度，从而使量化误差增大。因此，Q4_0 可以看作是一种在压缩率、实现复杂度和精度保持能力之间进行折中的基础量化方案：其优点是结构简单、压缩效率高、实现代价低；其缺点则在于量化粒度较粗，表示能力相对有限。

综上，Q4_0 是一种基于固定分块与统一缩放因子的 4 比特权重量化方法。其通过将每个量化块中的浮点权重映射到有限个离散整数编码，并利用

$$\hat{w}_i = d(q_i - 8)$$

实现近似重建，从而在显著降低存储开销的同时保留权重的主要数值特征。由于其形式简洁、实现直接，Q4_0 可作为低比特权重量化分析中的基础代表方法。

3 基于 SVD 的多核协同卸载计算方法建模

3.1 概述

在边侧设备本地推理场景中，设备可用计算资源通常会随系统运行状态动态变化。当边侧设备同时承担多任务处理、传感数据采集或后台服务时，CPU 利用率可能显著升高，进而导致神经网络推理时延增加、吞吐下降，甚至出现超时现象。对于部署在边侧设备上的深度学习模型而言，如何在资源波动条件下兼顾推理时延稳定性与输出精度，是本章关注的核心问题。

现有动态推理与近似推理方法大多面向特定模型结构展开设计。例如，一些方法主要针对卷积神经网络中的卷积层裁剪、通道裁剪或早退机制进行优化，另一些方法则更侧重于 Transformer 模型中的层跳过、注意力近似或分阶段执行。这类方法虽然在特定网络结构下能够取得较好效果，但往往缺乏统一的跨模型适配能力。当部署对象同时涉及传统 CNN 模型与 Transformer 模型时，通常需要为不同模型分别设计近似执行策略，导致方法复用性不足，系统适配成本较高。

针对上述问题，本章提出一种面向边侧设备本地推理的通用弹性补偿计算框架。该框架的核心思想是：首先将不同模型中的关键计算统一表述为矩阵乘法形式；随后利用奇异值分解对权重矩阵进行拆分，将原始计算划分为主贡献项与补偿项；再依据不同核心的实时负载状态，将主贡献项与补偿项映射到不同负载水平的核心上并行执行。其中，主贡献项对应模型中的高重要度计算，被分配到当前较为空闲、剩余算力较高的核心上执行，以保证推理输出的基础质量；补偿项对应较低重要度计算，被分配到当前负载较高或剩余算力较低的核心上执行，用于在资源允许的条件下进一步提升结果精度。若补偿项能够在规定时限内完成，则将其结果与主贡献项融合；若补偿项在给定时限内未能完成，则直接将其丢弃，仅保留主贡献项的输出结果，从而避免低重要度计算对整体推理时延造成显著影响。

在上述总体思路下，本章首先通过卷积矩阵化方法将传统 CNN 模型中的卷积层转化为等价矩阵乘法形式，从而与 Transformer 模型中的线性层纳入统一分析框架；然后在统一矩阵表示基础上，采用奇异值分解方法构建基于主贡献项与补偿项的任务卸载机制；最后结合候选卸载率、时延收益、精度损失以及相邻层约束，将时限约束下的弹性融合过程建模为一个受约束的全局优化问题，并进一步采用动态规划方法进行求解

3.2 基于 SVD 的卷积矩阵化乘法算子融合优化

对于 Transformer 模型中的线性投影层和前馈网络，由于其计算本身已经写成标准矩阵乘法形式，因此可直接纳入后续统一处理框架。对于传统 CNN 模型中的卷积层，其基本计算形式为卷积核在输入特征图上的滑动窗口运算，难以直接表示为标准矩阵乘法。为了使卷积层也能够采用与线性层一致的表示方式，并为后续基于奇异值分解的统一处理提供基础，本文引入 Expansion、Unfold 与 Fold 三个操作，将卷积运算转化为等价的矩阵乘法形式。

设输入特征张量为 X，卷积核张量为 W_c。首先定义 Expansion 操作，用于按照卷积滑窗访问关系，将输入特征张量中的局部感受野展开并重排为二维矩阵。记展开后的输入矩阵为

$$X_u = \mathrm{Expansion}(X)$$

其中，X_u 中的每一列或每一行对应输入特征在某一滑窗位置上的局部向量。通过该操作，卷积过程中按滑窗逐位置访问的输入结构被统一映射为规则的二维矩阵表示，从而将原本分散的局部访存过程转化为更适合统一处理的矩阵访存形式。

进一步定义 Unfold 操作，用于将卷积核张量 W_c 重排为二维矩阵。记卷积核矩阵化结果为

$$W_u = \mathrm{Unfold}(W_c)$$

该操作本质上是对卷积核参数排列方式进行变换，使原本以张量形式表示的卷积核转化为适于矩阵乘法计算的二维形式。经过 Expansion 和 Unfold 处理后，原始卷积运算即可改写为等价的矩阵乘法

$$X * W_c \Leftrightarrow X_u W_u$$

式（3-16）表明，卷积层虽然在实现上表现为滑动窗口卷积，但从数学表示上可以等价转写为输入展开矩阵与卷积核矩阵之间的乘法。这一变换使卷积层能够与 Transformer 中的线性层共享同一计算表示形式，也为后续使用统一的矩阵分解与任务卸载方法奠定了基础。

在得到矩阵乘法结果后，还需将其恢复为卷积输出张量形式。记矩阵乘法结果为 Y_u，则有

$$Y_u = X_u W_u$$

$$Y = \mathrm{Fold}(Y_u)$$

综合上述过程，卷积矩阵化与输出恢复的完整表达可写为

$$Y = \mathrm{Fold}\big(\mathrm{Expansion}(X)\,\mathrm{Unfold}(W_c)\big)$$

式（3-19）给出了卷积运算从张量域到矩阵域、再从矩阵域返回张量域的统一表达。其中，Expansion 用于构造输入局部感受野的矩阵表示，Unfold 用于完成卷积核的矩阵化，Fold 用于将矩阵乘法结果恢复为卷积输出张量。三者共同构成了卷积矩阵化的基本推导链路。

需要指出的是，本节引入 Expansion、Unfold 与 Fold 的目的，不仅在于建立卷积运算与矩阵乘法之间的数学等价关系，也在于从访存组织角度将原本不规则的滑窗访问转化为规则的矩阵访问模式，从而为后续基于统一矩阵形式的分解、调度与卸载提供条件。经过该步骤后，传统 CNN 模型中的卷积层与 Transformer 模型中的线性层均可统一写为矩阵乘法形式，为本章后续方法建立了共同的计算基础。

3.3 基于 SVD 的权重矩阵计算任务卸载方法

在完成卷积矩阵化之后，Transformer 模型中的线性层与 CNN 模型中的卷积层均可统一写为矩阵乘法形式

$$y = Wx$$

其中，W 表示权重矩阵，x 表示输入向量，y 表示输出结果。在此基础上，本文进一步采用奇异值分解对权重矩阵进行拆分，从而将原始计算划分为主贡献项与补偿项两部分。

对于任意权重矩阵 $W \in \mathbb{R}^{m\times n}$，均可表示为

$$W = U\Sigma V^T$$

其中，U 与 V 为列正交矩阵，Σ 为奇异值对角矩阵，设其奇异值满足

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

前若干个较大的奇异值通常承载了矩阵的主要结构信息，而较小奇异值对应的成分更多反映局部细节或次要补偿信息。因此，本文通过选定截断秩 k，将权重矩阵划分为主贡献项与补偿项两部分。

定义主贡献项为

$$W_{\text{main}}^{(k)} = U[:,1:k]\Sigma_{1:k}V[:,1:k]^T$$

定义补偿项为

$$W_{\text{tail}}^{(k)} = W - W_{\text{main}}^{(k)}$$

于是原始计算可写为

$$y = Wx = \left(W_{\text{main}}^{(k)} + W_{\text{tail}}^{(k)}\right)x$$

进一步定义

$$y_{\text{main}} = W_{\text{main}}^{(k)}x$$

$$y_{\text{tail}} = W_{\text{tail}}^{(k)}x$$

其中，y_{main} 表示主贡献项计算结果，决定了模型输出的基础质量；y_{tail} 表示补偿项计算结果，用于在主结果基础上进一步恢复被截断的信息。

在此基础上，本文构建面向边侧设备多核环境的任务卸载机制。设边侧设备共有 M 个计算核心，第 i 个核心当前利用率记为 $A_i \in [0,1]$。为更准确描述各核心当前可用于新任务的计算能力，定义第 i 个核心的有效剩余算力为

$$C_i = \phi_i(1 - A_i)$$

其中，ϕ_i 表示第 i 个核心的归一化计算能力；对于同构核心，可取 $\phi_i = 1$。于是，系统总有效剩余算力为

$$C_{\text{tot}} = \sum_{i=1}^{M} C_i$$

依据各核心当前有效剩余算力的差异，本文将边侧设备中的核心划分为两类逻辑执行集合：一类为剩余算力较高、当前较为空闲的核心集合；另一类为剩余算力较低、当前负载较高的核心集合。对于高重要度的主贡献项，系统将其映射到剩余算力较高的核心集合上执行，以提高关键计算的完成确定性并保证主结果的稳定输出；对于低重要度的补

偿项，系统则将其映射到剩余算力较低或当前负载较高的核心集合上执行，使其作为机会式任务参与并行推理过程。
需要强调的是，这里的差异化调度并不意味着严格的串行先后关系，而是一种基于核心负载状态的并行映射机制。主贡献项与补偿项可以在不同核心集合上同时启动执行，只是由于主贡献项承担基础输出责任，因此其被优先分配到更可靠的计算资源上，而补偿项则被放置在更易受系统波动影响的计算资源上。这样一来，即使系统整体负载升高或部分核心运行状态恶化，仍可优先保证主贡献计算顺利完成，从而维持模型输出的基本可用性。

需要进一步指出的是，本文并不为所有层统一指定固定截断秩，而是为每一层预先构建若干候选划分方案。不同划分方案对应不同规模的主贡献项与补偿项，其时延收益与精度代价也不同。通过这种方式，模型中的不同层能够在统一的表示框架下形成一组可选近似执行策略，为后续时限约束下的全局优化建模奠定基础。

3.4 时限约束下的弹性融合问题建模

在完成卷积矩阵化以及基于 SVD 的任务划分与多核并行执行机制设计后，仍需进一步回答一个关键问题：在边侧设备负载动态变化的条件下，如何为模型各层分配裁剪率，并在给定总允许超时量的条件下为各层补偿分支分配等待时间，从而在尽可能保持主贡献部分执行速度不低于模型原本最快请求速度的前提下，降低整体精度损失。为此，本文将时限约束下的弹性融合过程建模为一个双层耦合优化问题。
设模型共有 L 个可候选近似层。对于第 l 层，预先构建其候选裁剪率集合

$$Q_l = \{0, q_{l,1}, q_{l,2}, \dots, q_{l,N_l}\}$$

其中，0 表示该层不进行裁剪，其余 $q_{l,j}$ 表示不同程度的补偿划分比例。第 l 层选择裁剪率 q_l 后，其主贡献部分执行时间记为 $T_l^{\text{main}}(q_l)$，补偿部分引入的初始精度损失记为 $s_l(q_l)$。
设模型在边侧设备上的原始最快请求时间为 T_{base}，则各层裁剪率需满足主贡献部分总执行时间约束

$$\sum_{l=1}^{L} T_l^{\text{main}}(q_l) \le T_{\text{base}}$$

以尽可能保证主要计算部分按照原本最快请求速度完成。
进一步地，设系统允许的总超时量为 τ_{tot}。本文将该总超时预算在各层补偿分支之间进行分配，记第 l 层所分配到的超时时间为 τ_l，则有

$$\sum_{l=1}^{L} \tau_l \le \tau_{\text{tot}}$$

同时，考虑到某层裁剪量越大，其对应补偿部分规模越大，因此应分配更多等待时间，

本文令各层超时预算按裁剪量比例分配，即

$$\tau_l = \frac{w_l(q_l)}{\sum_{j=1}^{L} w_j\left(q_j\right)} \tau_{\text{tot}}$$

其中，$w_l(q_l)$ 为与裁剪量相关的权重函数。在最简单情形下，可取 $w_l(q_l) = q_l$。
此外，考虑到连续多层同时裁剪会导致模型精度快速下降，本文进一步引入相邻层约束。记

$$z_l = \begin{cases} 1, & q_l \neq 0 \\ 0, & q_l = 0 \end{cases}$$

则需满足

$$z_l + z_{l+1} \leq 1, \qquad l = 1,2,\dots,L-1$$

在上述约束下，本文希望通过联合确定各层裁剪率与补偿等待时间，在保证主贡献部分尽可能维持原始最快请求速度的前提下，最小化补偿未完成所带来的整体精度损失。于是，该问题可形式化为

$$\min_{q_1,\dots,q_L} \sum_{l=1}^{L} \tilde{s}_l\left(q_l, \tau_l\right)$$

s. t.

$$\sum_{l=1}^{L} T_l^{\text{main}}\left(q_l\right) \leq T_{\text{base}}$$

$$\sum_{l=1}^{L} \tau_l \leq \tau_{\text{tot}}$$

$$\tau_l = \frac{w_l(q_l)}{\sum_{j=1}^{L} w_j\left(q_j\right)} \tau_{\text{tot}}$$

$$q_l \in Q_l, \qquad l = 1,2,\dots,L$$

$$z_l + z_{l+1} \leq 1, \qquad l = 1,2,\dots,L-1$$

其中，$\tilde{s}_l(q_l, \tau_l)$ 表示第 l 层在裁剪率 q_l 与超时预算 τ_l 共同作用下的最终有效精度损失。由于该问题同时具有分层决策特征、局部相邻约束以及总超时预算约束，因此后续可进一步采用动态规划方法进行求解。

3.5 不同模型类型下的协同方向选择策略

前文已经分别给出了卷积矩阵化表示方法、基于奇异值分解的主贡献项与补偿项划分方法，以及时限约束下的弹性融合建模框架。需要进一步指出的是，虽然上述方法在数学形式上能够统一作用于 Transformer 模型中的线性层和传统 CNN 模型中的卷积层，但两类模型在实际部署场景中的默认运行位置、资源瓶颈分布以及协同方向并不完全相同。因此，在具体系统设计中，需要针对不同模型类别选择不同的协同推理路径。

对于 Transformer 类模型，尤其是参数规模较大的大语言模型，其主要计算负载通常集中在边侧设备上完成。原因在于该类模型参数规模大、主路径推理开销高，对计算资源和内存资源要求较高，而边侧设备通常具备更强的算力与更稳定的运行环境，更适合承担模型推理过程中的主体计算任务。因此，对于 Transformer 类模型，本文采用“边侧优先、本地裁剪、必要时卸载到端侧”的协同思路。具体而言，首先按照式（3-20）至式（3-27）对关键矩阵乘法层进行统一表示与主贡献项 — 补偿项划分，并依据式（3-28）和式（3-29）刻画边侧设备多核资源状态；在边侧本地优先执行主贡献路径的基础上，若根据第 3 章中时限约束建模无法找到满足实时性的本地可行解，则进一步从模型最后几层开始向端侧设备卸载计算任务，以恢复整体推理的可行性。由此，Transformer 模型对应的协同方向表现为 边 → 端。

与之不同，传统 CNN 模型在很多实际应用中通常默认运行在手机等端侧设备上。这类模型规模相对较小，且更常服务于图像分类、检测、识别等端侧智能任务，因此其初始部署位置往往就是手机端。当手机端 CPU 占用较低时，可直接在本地完成推理；当手机端负载上升时，则可首先利用前文卷积矩阵化方法将卷积层转写为矩阵乘法形式，再依据式（3-21）至式（3-27）对关键计算层进行主贡献项与补偿项划分，在手机端内部优先执行主贡献部分，并利用弹性补偿机制在时限允许条件下尽可能恢复精度。若在手机端本地仍无法满足实时约束，则进一步从模型最后几层开始，将部分后层任务卸载到边侧设备执行，以借助边侧更强的算力恢复实时推理能力。由此，传统 CNN 模型对应的协同方向表现为 端 → 边。

从上述分析可以看出，本文方法的统一性体现在：无论是 Transformer 模型还是 CNN 模型，其关键计算都可归一到矩阵乘法形式，并利用相同的奇异值分解方法构建主贡献项与补偿项划分，同时在时限约束下通过裁剪率分配与弹性融合机制实现近似执行。其差异性则体现在：两类模型的默认部署位置不同，导致系统负载瓶颈的主要出现位置不同，进而使协同方向发生变化。对于主要部署在边侧设备上的 Transformer 类模型，协同方向为边 → 端；对于主要部署在端侧设备上的传统 CNN 模型，协同方向为端 → 边。

因此，本文所提出的方法并非仅适用于单一固定方向的协同推理，而是一种与模型类别和默认部署位置相关联的双向协同框架。在后续联合实时调度策略中，系统将根据模型类型、当前主要执行设备的负载状态以及本地近似调度是否存在可行解，动态决定采用边 → 端还是端 → 边的协同路径，从而实现面向不同模型与不同资源瓶颈位置的统一支持。

4 面向无原始数据场景的协同策略精度评估算法

4.1 概述

在云边协同推理场景中，模型切分与边缘侧压缩策略的选择通常需要评估其对模型精度的影响。传统方法大多依赖原始训练数据或验证数据集进行测试，但在隐私受限、数据不可回传以及模型快速迭代的实际部署环境中，这类条件往往难以满足。为此，本文参考 Synthetic Data Generator 的思想，设计一种面向无原始数据场景的协同策略精度评估方法。

该方法不直接依赖真实样本，而是通过模型自身的输出特性反向构造合成输入，使模型对指定类别产生高置信度响应。利用这些合成样本，可以在无原始数据条件下估计协同策略实施后的精度变化，从而为模型切分和边缘侧压缩提供依据。

4.2 问题定义与符号约定

设完整神经网络模型记为 $F(X)$，输入样本记为 X，模型输出类别数为 C。对于给定目标类别 $\hat{c}$，本文希望构造一个合成输入 X，使模型在该类别上的预测概率尽可能高。为统一描述，记模型对输入 X 的第 c 类输出 logit 为 $F(X)_c$，对应 softmax 概率如下：

$$P(c \mid X) = \frac{e^{F(X)_c}}{\sum_{v=1}^{C} e^{F(X)_v}}$$

4.3 损失函数设计

为了引导输入向目标类别方向迭代优化，本文采用交叉熵损失函数作为输入空间拟合的优化目标。设目标类别为 $\hat{c}$，对应独热标签向量 t，当 $c = \hat{c}$ 时 $t_c = 1$，否则 $t_c = 0$，则损失函数定义为：

$$L(X) = -\sum_{c=1}^{C} t_c \log\left(\frac{e^{F(X)_c}}{\sum_{v=1}^{C} e^{F(X)_v}}\right)$$

该损失函数能够度量模型输出分布与目标类别分布之间的差异。当输入经过优化后，若模型在目标类别上的输出概率不断增大，则损失持续减小。由于该过程仅依赖模型参数和目标类别，不依赖真实数据，因此适用于无原始数据场景下的协同策略精度评估。

4.4 基于梯度下降的输入空间拟合算法

在给定损失函数后，本文采用梯度下降方法在输入空间中更新样本。首先随机初始化输入 $X^{(0)}$，然后计算损失函数关于输入的梯度，并按照梯度下降方向不断迭代，更新公式为：

$$X^{(k+1)} = X^{(k)} - \eta \cdot \frac{\partial L\left(X^{(k)}\right)}{\partial X^{(k)}}$$

经过多轮迭代后，输入将从随机噪声逐步演化为能够使模型在目标类别上产生高置信度预测的合成样本。该样本可作为代理输入，用于评估不同协同部署策略下模型精度的变化。

4.4.1面向 Transformer 架构模型的输入空间拟合方法

对于 Transformer 架构模型，目标类别 $\hat{c}$ 可理解为词表中的目标 token 索引。输入空间拟合时，从随机初始化的输入表示出发，通过最小化交叉熵损失，不断调整输入表示，使模型在目标 token 上输出更高概率。

这种方法本质上是在表示空间中搜索能够激发目标输出的输入特征。生成结果未必对应真实自然语言序列，但能够反映模型在目标 token 方向上的内部响应特性，因此可以作为无原始数据条件下评估协同策略的代理样本。

4.4.2面向卷积架构模型的输入空间拟合方法

对于卷积神经网络，目标类别 $\hat{c}$ 直接对应图像分类任务中的类别编号。算法首先随机初始化一幅输入图像，再通过反向传播计算损失函数对像素的梯度，并持续更新图像像素值，使模型在目标类别上的输出概率不断提高。

经过若干次迭代后，初始随机噪声会逐步转化为能够激发卷积网络特定类别响应的合成图像。尽管该图像不一定具有真实样本的自然语义，但其能够表征模型的判别行为，因此可以用于无原始数据场景下协同策略的精度评估。

5 基于负载感知的边端协同联合实时调度策略

5.1 概述

在前述章节中，本文已经给出了基于卷积矩阵化的统一计算表示方法、基于奇异值分解的主贡献项与补偿项划分方法，以及面向边侧设备本地推理的弹性融合问题建模。上述方法主要解决的是：在边侧设备本地资源动态变化的条件下，如何通过对模型关键计算进行裁剪与分解，在保证主路径时延可控的前提下，尽可能维持模型输出精度。
然而，在实际运行过程中，边侧设备所面临的负载状态可能进一步恶化。当 CPU 利用率持续升高、可用核心算力显著下降时，即便采用前述的层内裁剪与弹性融合机制，也可能出现如下情况：对模型各候选层遍历所有可选裁剪率后，仍不存在一组可行策略能够使主贡献部分在目标时间内完成推理。此时，仅依赖边侧本地近似执行已不足以满足实时性要求，需要进一步引入端侧设备参与协同推理，将部分后层计算卸载到手机等端侧设备上执行，从而扩展整体可用计算资源，恢复系统的实时推理能力。

基于此，本章进一步提出一种面向负载感知场景的边端协同联合实时调度策略。该策略将第 3 章中的层内裁剪、超时预算分配与本章中的跨设备任务迁移结合起来，形成“边侧优先、本地近似、必要时后层卸载”的联合调度框架。整体而言，该策略分为两个层次：首先，在边侧本地资源仍可支撑任务按时完成时，优先利用动态规划方法在各候选层之间分配裁剪率与超时预算，以最小化整体精度损失；其次，当上述动态规划问题无可行解时，说明仅依赖边侧本地裁剪已无法满足实时要求，此时从模型后层开始向端侧逐步卸载计算任务，直到找到能够满足时限约束的最小卸载层集合，从而在尽可能减少跨设备传输与结构扰动的前提下恢复可行调度方案。

需要特别指出的是，端侧设备虽然能够提供额外算力，但其执行环境与边侧设备并不完全一致。本文中，手机端采用 4bit 权重量化方式进行推理计算，因此卸载到手机端的任务除了承受分割与中间特征传输带来的影响外，还会额外引入量化误差。随着卸载层数增加，参与 4bit 量化执行的网络层范围也随之扩大，模型整体精度退化风险将进一步上升。因此，在高负载场景下引入边端协同的目标并非简单地“尽可能多地将任务卸载到手机端”，而是在恢复实时性的前提下，尽可能减少卸载到手机端的层数与计算范围，以抑制由于低比特量化所带来的额外精度损失。

本章将在此基础上，首先给出基于动态规划的计算任务分配策略及其求解流程；随后进一步讨论当本地近似调度无解时，如何在高负载场景下进行边端协同推理，并给出从第几层开始卸载以及如何求解最小可行卸载层集合的方法。

5.2 基于动态规划的计算任务分配策略

在第 3 章中，本文已经将时限约束下的弹性融合过程建模为一个带局部相邻约束与总超时预算约束的优化问题。该问题的核心在于：在给定边侧设备当前负载状态的条件下，为模型各候选层选择合适的裁剪率，并在总允许超时量范围内分配各层补偿等待时间，从而在尽可能维持主贡献路径执行速度的前提下，最小化整体精度损失。本节对该问题的动态规划求解过程作进一步展开，并在此基础上给出各层超时任务的预算分配方

法。

5.2.1 核心空闲率建模与核心集合枚举

设当前设备共有 M个 CPU 核心，第 i个核心当前利用率记为 $A_i \in [0,1]$。为刻画该核心当前可用于新推理任务的计算能力，沿用第 3 章中的有效剩余算力定义：

$$C_i = \phi_i(1 - A_i)$$

其中，ϕ_i 表示第 i个核心的归一化计算能力。对于同构核心，可取 $\phi_i = 1$；对于异构核心，ϕ_i 可用于反映不同核心之间的基础性能差异。
为了将 CPU 核心空闲率纳入调度决策，本文首先根据有效剩余算力 C_i对所有核心进行降序排序，得到核心序列

$$C_{\pi_1} \geq C_{\pi_2} \geq \cdots \geq C_{\pi_M}$$

其中，π_j 表示排序后第 j个核心的原始编号。有效剩余算力越高，说明该核心当前越适合承担主贡献部分的关键计算。
在此基础上，本文枚举主贡献部分所使用的核心数量 p。对于给定 p，定义主贡献核心集合为

$$\mathcal{C}_{\text{major}}^{(p)} = \{\pi_1, \pi_2, \dots, \pi_p\}$$

定义补偿核心集合为

$$\mathcal{C}_{\text{minor}}^{(p)} = \{\pi_{p+1}, \pi_{p+2}, \dots, \pi_M\}$$

其中，$\mathcal{C}_{\text{major}}^{(p)}$ 用于执行主贡献项，$\mathcal{C}_{\text{minor}}^{(p)}$ 用于执行尾部补偿项。随着 p增大，更多空闲核心被分配给主贡献部分，主路径执行稳定性增强；但可用于补偿计算的核心数量减少。反之，较小的 p会为补偿部分保留更多核心，但主贡献部分可用核心减少，可能导致主路径时延增加。
特别地，当

$$p = M$$

时，有

$$\mathcal{C}_{\text{major}}^{(M)} = \{\pi_1, \pi_2, \dots, \pi_M\}$$

$$\mathcal{C}_{\text{minor}}^{(M)} = \emptyset$$

此时全部核心均用于主贡献部分，不再划分补偿核心集合。该模式对应完整主路径执行，可视为“不裁剪、不卸载”的本地执行方案。因此，本文规定当 $p = M$时，各层裁剪率均为

$$q_l = 0, l = 1,2, \dots, L$$

即若所有核心均参与主路径计算，则不再启用 SVD 尾部补偿任务

5.2.2固定核心划分模式下的动态规划建模

在第 3 章中，本文已经根据各核心当前有效剩余算力，将边侧设备中的核心划分为主贡献项执行核心集合和补偿项执行核心集合。其中，主贡献项应优先映射到剩余算力较高、当前较为空闲的核心上执行，以保证主路径输出的稳定性；补偿项则可映射到剩余算力较低或当前负载较高的核心上执行，使其作为机会式计算任务参与推理过程。但是，在原有动态规划建模中，核心划分并未显式进入求解过程，动态规划仅针对各层裁剪率进行搜索。因此，当 CPU 核心整体较为空闲时，算法仍可能选择非零裁剪率，从而引入不必要的精度损失。

为解决上述问题，本文在动态规划求解之前，首先引入基于核心空闲率的 major-minor 核心划分枚举机制。该机制并不改变后续动态规划的状态定义，而是在动态规划外层枚举不同的核心划分策略。对于每一种核心划分策略，分别执行一次动态规划求解；最终在所有核心划分策略对应的可行解中，选择整体效果最优的方案。这样既能将 CPU 当前空闲状态纳入决策，又能避免将每个核心的状态直接加入动态规划状态空间所导致的复杂度膨胀。

设当前设备共有 M个 CPU 核心，第 i个核心当前利用率为 $A_i \in [0,1]$。沿用第 3 章中有效剩余算力的定义，记第 i个核心当前可用于新任务的有效剩余算力为

$$C_i = \phi_i(1 - A_i)$$

其中，ϕ_i 表示第 i个核心的归一化计算能力。对于同构核心，可取 $\phi_i = 1$；对于异构核心，ϕ_i 可用于反映不同核心之间的基础性能差异。

随后，根据 C_i对所有核心进行降序排序，得到核心序列

$$C_{\pi_1} \geq C_{\pi_2} \geq \cdots \geq C_{\pi_M}$$

其中，π_j 表示排序后第 j个核心的编号。有效剩余算力越高，说明该核心当前越适合承担主贡献部分的关键计算。

在排序结果基础上，本文枚举主贡献部分所使用的核心数量 p。对于给定 p，定义 major 核心集合为

$$\mathcal{C}_{\text{major}}^{(p)} = \{\pi_1, \pi_2, \ldots, \pi_p\}$$

定义 minor 核心集合为

$$\mathcal{C}_{\text{minor}}^{(p)} = \{\pi_{p+1}, \pi_{p+2}, \ldots, \pi_M\}$$

其中，$\mathcal{C}_{\text{major}}^{(p)}$ 用于执行主贡献项，$\mathcal{C}_{\text{minor}}^{(p)}$ 用于执行尾部补偿项。随着 p增大，更多空闲核心被分配给主贡献部分，主路径的执行稳定性提高；但可用于补偿部分的核心数量减少。反之，较小的 p会为补偿计算保留更多核心，但主贡献部分可用核心减少，可能

导致主路径执行时间增加。
特别地，当

$$p = M$$

时，有

$$\mathcal{C}_{\text{major}}^{(M)} = \{\pi_1, \pi_2, \dots, \pi_M\}$$

$$\mathcal{C}_{\text{minor}}^{(M)} = \emptyset$$

此时全部核心均被分配给主贡献部分，不再设置补偿核心集合。该模式对应完整本地执行策略，即不进行裁剪与补偿卸载。由此，CPU 较为空闲时，若完整执行已经能够满足目标时限，算法可以自然选择 $p = M$对应的模式，从而避免不必要的精度损失。
对于每一个核心划分模式 p，本文分别执行一次后续动态规划求解。需要指出的是，在不同核心划分模式下，同一层、同一裁剪率对应的主贡献部分执行时间可能不同。因此，在模式 p下，第 l层采用裁剪率 q_l时的主贡献部分执行时间记为

$$T_l^{\text{main},(p)}(q_l)$$

补偿部分执行时间记为

$$T_l^{\text{tail},(p)}(q_l)$$

其中，$T_l^{\text{main},(p)}(q_l)$ 由 major 核心集合 $\mathcal{C}_{\text{major}}^{(p)}$决定，$T_l^{\text{tail},(p)}(q_l)$ 由 minor 核心集合 $\mathcal{C}_{\text{minor}}^{(p)}$决定。后续动态规划的状态定义和转移方程保持不变，只需在固定模式 p下使用对应的 $T_l^{\text{main},(p)}(q_l)$作为主路径时间代价。

5.2.3状态定义

设模型共有 L 个可候选近似层。对于第 l 层，预先构建其候选裁剪率集合

$$Q_l = \{0, q_{l,1}, q_{l,2}, \dots, q_{l,N_l}\}$$

其中，0 表示该层不进行裁剪，其余 $q_{l,j}$ 表示不同程度的裁剪率。对于给定裁剪率 q_l，可离线测得以下量：主贡献部分执行时间 $T_l^{\text{main}}(q_l)$、初始精度损失 $s_l(q_l)$ 以及补偿部分规模权重函数 $w_l(q_l)$。
其中，精度损失$s_l(q_l)$定义为原始权重矩阵与裁剪后主贡献矩阵之间的矩阵二范数距离：设第 l层的原始权重矩阵为 W_l，在裁剪率 q_l下对应的主贡献项记为 $W_l^{\text{main}}(q_l)$，则有

则有

$$s_l(q_l) = \| W_l - W_l^{\text{main}}(q_l) \|_2$$

由于被裁剪部分正对应于该层的尾部补偿项，记

$$W_l^{\text{tail}}(q_l) = W_l - W_l^{\text{main}}(q_l)$$

则上式也可写为

$$s_l(q_l) = \| W_l^{\text{tail}}(q_l) \|_2$$

设模型原始最快请求时间为 T_{base}，系统总允许超时量为 τ_{tot}。则动态规划的目标是在满足主路径总执行时间不超过 T_{base}、总超时预算不超过 τ_{tot}、任意相邻两层不能同时进行裁剪的前提下，最小化整体有效精度损失。

为此，引入二值变量

$$z_l = \begin{cases} 1, & q_l \neq 0 \\ 0, & q_l = 0 \end{cases}$$

并满足

$$z_l + z_{l+1} \leq 1, \qquad l = 1,2,\dots,L-1$$

对于固定核心划分模式 p，定义动态规划状态为

$$\text{dp}^{(p)}[l][t][b]$$

其中，l 表示当前已处理到第 l层，t 表示当前累计主路径执行时间，$b \in \{0,1\}$ 表示第 l层是否执行了裁剪。状态 $\text{dp}^{(p)}[l][t][b]$的含义为：在核心划分模式 p下，处理完前 l层、累计主路径执行时间为 t、且当前层裁剪状态为 b时，能够达到的最小初始精度损失。

这里的 $\text{dp}^{(p)}$与原动态规划状态定义的含义保持一致，只是增加了上标 (p)，用于表示当前动态规划是在第 p种 major-minor 核心划分模式下执行。也就是说，核心划分并不作为动态规划状态变量参与转移，而是作为动态规划外层枚举参数存在。

5.2.4状态转移方程推导

对于第 l 层，枚举其候选裁剪率 $q \in Q_l$。若该层选择裁剪率 q，则对应主贡献部分执行时间增加 $T_l^{\text{main}}(q)$，精度损失增加 $s_l(q)$，并有当前裁剪状态

$$b' = \begin{cases} 1, & q \neq 0 \\ 0, & q = 0 \end{cases}$$

若前一层状态为 b，则只有在满足

$$b + b' \leq 1$$

时，该候选动作才是可行的。于是在核心划分模式 p下，状态转移可写为:

$$\text{dp}^{(\text{p})}[l][t'][b'] = \min\left(\text{dp}^{(\text{p})}[l][t'][b'], \text{dp}^{(\text{p})}[l-1][t][b] + s_l(q)\right)$$

其中：

$$t' = t + T_l^{\text{main},(p)}(q)$$

动态规划的初始条件为

$$\text{dp}[0][0][0] = 0$$

其余状态初始化为无穷大。在完成所有 L 层的状态转移后，若存在某个终态满足 $t \leq T_{\text{base}}$，则说明边侧本地裁剪策略存在可行解；反之，则说明在当前负载条件下，仅依赖边侧本地裁剪无法保证模型按时完成推理，此时需转入 5.2 节所述的边端协同推理策略。

5.2.5任务超时分配策略设计

在通过动态规划确定各层裁剪率之后，还需进一步为各层补偿分支分配等待时间。设第 l 层分配到的超时预算为 τ_l，则需满足总预算约束

$$\sum_{l=1}^{L} \tau_l \leq \tau_{\text{tot}}$$

结合第 3 章中的建模假设，本文令各层超时预算按裁剪量加权分配，即

$$\tau_l = \frac{w_l(q_l)}{\sum_{j=1}^{L} w_j\left(q_j\right)} \tau_{\text{tot}}$$

其中，$w_l(q_l)$ 为与第 l 层裁剪量相关的权重函数。在最简单情况下，可取

$$w_l(q_l) = q_l$$

进一步地，考虑到不同层即使裁剪量相同，其补偿计算复杂度也可能不同，因此上述权重函数也可推广为

$$w_l(q_l) = \alpha_l q_l$$

其中，α_l 表示第 l 层单位裁剪量对应的补偿复杂度修正系数。由此，超时预算分配策略不仅能够反映裁剪规模，还可进一步体现不同层补偿计算复杂度的差异。

5.2.6求解流程

综合上述内容，基于动态规划的计算任务分配策略可归纳为如下步骤：

1. 对模型各候选层预先构建裁剪率集合，并测得对应的主路径执行时间、精度损失与补偿权重；
2. 在给定当前负载状态下，根据目标最快请求时间 T_{base} 建立动态规划状态空间；
3. 逐层枚举候选裁剪率，结合相邻层不可连续裁剪约束完成状态转移；

4. 在所有满足主路径时限约束的终态中，选取整体精度损失最小的一组裁剪率作为最优本地调度结果；

5. 根据所选裁剪率及总允许超时量 τ_{tot}，按裁剪量比例为各层分配补偿等待时间。

由此可见，本节所给出的动态规划求解策略本质上是在边侧设备本地资源约束下，对“主路径保速”和“补偿路径回收精度”进行联合优化。只有当该本地优化过程无可行解时，才有必要进一步引入跨设备协同执行，这也是下一节所要讨论的内容。

5.3 面对高负载场景的边端协同推理

在实际运行过程中，边侧设备所承受的负载可能持续升高，以致仅依赖边侧本地裁剪已无法满足实时推理要求。具体而言，当第 5.1 节中的动态规划过程无可行解时，说明对于所有可候选层及其可能的裁剪率组合，都不存在一种本地执行策略能够使主路径在目标时间 T_{base} 内完成推理。此时，仅通过继续加大本地裁剪程度已无法恢复可行性，需要进一步引入端侧设备参与协同推理，将部分后层计算任务卸载到手机等端侧设备上执行。

需要特别指出的是，端侧设备虽然能够提供额外算力，但其执行环境与边侧设备并不完全一致。本文中，手机端采用 4bit 权重量化方式进行推理计算，因此卸载到手机端的任务除了承受分割与中间特征传输带来的影响外，还会额外引入量化误差。随着卸载层数增加，参与 4bit 量化执行的网络层范围也随之扩大，模型整体精度退化风险将进一步上升。因此，在高负载场景下引入边端协同的目标并非简单地“尽可能多地将任务卸载到手机端”，而是在恢复实时性的前提下，尽可能减少卸载到手机端的层数与计算范围，以抑制由于低比特量化所带来的额外精度损失。

基于此，本文在本地动态规划无可行解时，采取如下协同策略：从模型后层开始向端侧逐步卸载计算任务，直到找到一组最小的后层卸载集合，使得边侧剩余前层任务能够在本地近似裁剪与弹性融合机制支持下满足实时约束。这里“最小”不仅意味着卸载层数尽可能少、跨设备传输代价尽可能低，更意味着进入手机端 4bit 量化执行范围的网络层尽可能少，从而降低量化误差对最终推理精度的影响。

5.3.1高负载场景下的协同基本思路

在边端协同推理中，边侧设备通常承担前层计算，端侧设备承担后层计算，二者之间通过中间特征传输实现串联执行。考虑到模型后层更接近输出端，其对中间表示的依赖更集中，且从后向前逐步卸载能够尽量减少对整体结构的扰动，因此本文采取如下基本策略：当本地动态规划无解时，从模型最后几层开始向端侧逐步卸载计算任务，直到找到一组最小的后层卸载集合，使得边侧剩余前层任务能够在本地近似裁剪与弹性融合机制支持下满足实时约束。

基于上述分析，本节所研究的核心问题可以表述为：在本地动态规划无可行解时，如何确定从第几层开始向端侧卸载，使得边侧剩余计算重新变为可行，同时卸载到端侧的层数最少，并尽可能降低手机端 4bit 量化带来的额外精度损失。

5.3.2 边端分割点选择问题建模

设模型共有 L 层。记边端分割点为 m，其含义为：第 1 至第 m 层仍在边侧设备上执行，第 $m+1$ 至第 L 层卸载到端侧设备执行。于是，当分割点为 m 时，边侧只需对前 m 层进行本地裁剪与调度，而后续层由端侧负责完成。设对应的中间特征传输时间为 $T_{\text{tx}}(m)$，端侧后层执行时间为 $T_{\text{end}}(m)$。则整体推理时间可表示为

$$T_{\text{total}}(m) = T_{\text{edge}}(m) + T_{\text{tx}}(m) + T_{\text{end}}(m)$$

其中，$T_{\text{edge}}(m)$ 表示边侧前 m 层在最优裁剪调度下的最小可行主路径执行时间。

对给定分割点 m，边侧前 m 层的调度问题仍可通过第 5.1 节所述动态规划方法求解。若存在某组前层裁剪策略使得

$$T_{\text{total}}(m) \le T_{\text{req}}$$

其中 T_{req} 表示系统允许的最大实时响应时限，则说明该分割点对应的边端协同方案是可行的。由于手机端采用 4bit 权重量化，端侧执行时间 $T_{\text{end}}(m)$ 虽然可能因低比特计算而得到压缩，但其对应的精度损失也不可忽略。记从第 $m+1$ 层到第 L 层卸载到端侧所引入的额外量化损失为

$$\Delta_{\text{quant}}(m)$$

则一般而言，卸载层数越多，$\Delta_{\text{quant}}(m)$ 越大。因此，在所有满足实时约束的分割点中，应优先选择使卸载层数最少、对应量化损失最小的方案。

基于此，本文将边端分割点选择问题定义为：在所有满足

$$T_{\text{total}}(m) \le T_{\text{req}}$$

的可行分割点中，选择最大的 m。由于 m 越大，意味着保留在边侧本地执行的层数越多、卸载到手机端的层数越少，因此这一选择等价于找到最少的可满足实时性的卸载后层集合。形式上，可写为

$$m^* = \max\{\, m \mid T_{\text{total}}(m) \le T_{\text{req}} \,\}$$

对应地，最优卸载层集合为

$$\{m^*+1, m^*+2, \dots, L\}$$

5.3.3 最优化求解方案

为求得最小可行卸载层集合，本文采用一种自后向前的逐步搜索策略。具体过程如下：

1 先假设仅卸载最后 1 层，即令分割点为 $m = L - 1$，并对前 m 层重新执行第 5.1 节中的动态规划求解，计算边侧最优主路径执行时间；

2 结合对应的中间特征传输时间与端侧执行时间，判断当前分割点是否满足整体实时约束；

3 若满足，则当前方案即为最优方案，因为此时卸载到手机端的层数最少；

4 若不满足，则继续将分割点向前移动一层，即再多卸载 1 层到手机端，重复上述判断过程；

5 直到找到第一个满足实时约束的分割点为止。

为了进一步统一表述，可将该过程描述为如下求解问题：

$$\min_{m} (L - m)$$

s.t.

$$T_{\text{total}}(m) \leq T_{\text{req}}$$

其中，目标函数 $L - m$ 表示卸载到手机端的层数。该优化问题的最优解即为满足实时约束条件下的最小卸载层集合方案。由于目标函数关于分割点 m 单调，因此采用自后向前逐层搜索的方式即可得到全局最优解。

5.3.4边端协同场景联合调度总流程

综合 5.1 节与本节内容，本文所提出的基于负载感知的边端协同联合实时调度策略可归纳为如下流程：

1. 首先获取边侧设备当前负载状态，并据此建立各候选层在当前条件下的裁剪执行代价；
2. 在边侧本地执行第 5.1 节所述动态规划求解，判断是否存在满足最快请求时间约束的本地裁剪方案；
3. 若存在可行解，则采用本地裁剪与超时预算分配策略完成推理；
4. 若不存在可行解，则进入边端协同模式，从模型最后一层开始逐步向前搜索分割点；

5. 对每一个候选分割点，重新求解边侧前层的本地动态规划问题，并结合中间特征传输时间与手机端执行时间判断整体是否可行；
6. 选择第一个满足实时约束的分割点作为最优边端协同方案，并将该分割点之后的层卸载到手机端执行；
7. 在所有可行方案中，由于该方案对应手机端卸载层数最少，因此同时也能较好抑制由于 4bit 权重量化所引入的附加精度损失。

由此，本文构建了一种从边侧本地弹性调度自然过渡到边端协同推理的联合实时调度框架：在负载较轻时，优先通过边侧本地裁剪与弹性融合完成推理；在负载过高、本地调度无解时，再引入端侧设备参与协同计算，并通过最小可行卸载层集合搜索尽可能减少进入手机端量化执行的层数。该策略既保证了系统在高负载场景下的实时可行性，又兼顾了跨设备协同所带来的额外精度损失控制。

6 实验平台与关键模块实现

6.1 基础推理引擎：Llama.cpp

6.2 高效自研算子与算子融合方法设计与实现

在第 3 章中,本文已经从数学上给出了基于奇异值分解的主贡献项与补偿项划分方法。对于分解后的线性层，其计算可以表示为两段连续过程：首先由输入向量与分解矩阵 V相乘，得到中间结果；随后再由中间结果与分解矩阵 U相乘，得到最终输出。形式上可写为

$$tmp = xV, y = tmpU$$

从数学角度看，上述过程完全等价于对原始权重矩阵进行低秩近似后的线性变换。然而，在实际推理运行时中，如何高效地执行这两段计算，是本文实现层面需要解决的关键问题。

6.2.1 直接复用原有矩阵乘法算子的局限性

一种最直接的实现思路，是将 SVD 分解后的两段计算分别交由 llama.cpp 原有的通用矩阵乘法算子执行，即先执行一次 xV，再执行一次 $tmpU$。从结果上看，这样的实现方式当然是正确的，因为它仍然严格遵循 SVD 分解后的数学表达。

但问题在于，通用矩阵乘法算子是为**一般矩阵形状、一般批量规模和一般张量布局**设计的。为了支持这些通用场景，其执行路径通常会包含较为完整的准备与调度逻辑，例如数据布局处理、循环分块、临时缓冲组织以及线程工作划分等。这些机制在大批量、大

矩阵计算场景下是必要的，因为它们有助于提高整体吞吐；然而在本文所关注的大语言模型 decode 阶段，输入通常已经退化为**单 token 对应的一条向量**，此时每次调用的问题规模很小，真正敏感的往往不是纯乘加运算，而是算子外层的固定组织成本。
若直接使用原有的矩阵乘法算子来实现 SVD 两阶段计算，则会带来两方面额外开销。第一，原本一层线性变换只需要承担一次矩阵乘法调度，现在却需要先后执行两次独立的通用矩阵乘法，从而使准备开销、调度开销与线程划分开销被重复承担。第二，两阶段之间会显式产生中间结果 tmp，这意味着除了两次计算本身外，还会增加中间结果的写回、再读取和额外访存组织成本。对于 decode 阶段这种“小输入、高频调用”的场景，这类额外开销会在大量重复调用中被持续放大，从而使整体效率明显下降。
因此，虽然直接复用 llama.cpp 原有矩阵乘法算子在数学上没有问题，但在运行时性能上并不适合本文的 SVD 场景。为了尽量减少两阶段独立调度所带来的额外成本，本文将 xV与 $tmpU$两段计算合并到一个专用执行路径中，并为其设计专门的 SVD 算子。其本质目标并不是改变原有计算结果，而是使两阶段计算只承担一次专用算子进入与调度的固定开销，从而更好适应单 token decode 热路径的执行特征。

6.2.2 采用三层循环实现的局限性

在明确需要构造专用 SVD 算子之后，仍然存在另一个问题：专用算子内部究竟如何实现。一种看似直接的方式，是按照最朴素的矩阵乘法定义，在算子内部采用传统三层循环完成两阶段乘加过程。例如，对于第一阶段 $tmp = xV$，可以按输出维度逐个累加输入向量与矩阵列之间的乘积；对于第二阶段 $y = tmpU$，也可以用同样方式逐项完成求和。这样做在逻辑上最容易理解，也最符合线性代数公式的直接展开形式。
但是，这种传统三层循环实现方式的主要问题在于：它虽然消除了通用矩阵乘法外层的部分调度开销，却没有真正发挥现代处理器的并行能力。三层循环本质上仍然是**标量粒度**的实现，即每次只完成一个元素的乘法与加法，再继续处理下一个元素。对于 CPU 而言，这意味着大量独立的乘加操作将逐个串行完成，难以充分利用宽向量寄存器、流水线并行与底层高效乘加指令。

进一步地，在 decode 场景中，虽然单次问题规模较小，但由于调用频率极高，算子内部的每一处低效实现都会被迅速累积放大。如果专用 SVD 算子只是简单地用三层 for 循环重写，那么它虽然避开了两次通用 mul_mat 的调度壳，但仍然会在真正的乘加计算阶段损失大量性能。换言之，这样的实现只能解决“外层调度太重”的问题，却无法解决“内层计算不够快”的问题。
因此，仅仅自己写一个专用算子还不够，关键还在于：**专用算子内部必须采用比传统标量循环更适合硬件执行的计算组织方式**。这也是本文进一步引入向量点积内核的根本原因。

6.2.3基于向量点积的高效矩阵乘法算子设计

6.2.4面向 SVD 计算的矩阵乘法算子融合（428，改内容）

在构造 SVD 专用算子的过程中，仅仅避免两次通用矩阵乘法的重复调度还不够。对于 decode 阶段而言，单次问题规模虽然较小，但调用频率极高，因此若希望进一步提升算子执行效率，还必须尽可能提高底层硬件的并行利用率。换言之，专用算子的优化目标不仅是减少通用调度开销，还应尽可能使其内部计算形式与处理器所提供的底层并行执行能力相匹配，从而最大化硬件吞吐。

现代 CPU 的高性能执行能力，很大程度上来源于向量化指令支持。与传统标量指令一次只处理一个数据不同，向量化指令可以在一个时钟周期内并行处理一组数据，从而显著提高乘加运算的吞吐效率。在本文相关实验平台中，这种能力具体体现为底层向量化乘加支持：对于浮点路径，可通过向量寄存器同时完成多组浮点乘加；对于低比特量化路径，则可进一步利用面向整型点积或整型矩阵乘的硬件能力完成更高密度的乘加累积。因此，若要真正发挥处理器的执行并行度，关键就在于把待计算问题组织成能够直接映射到底层向量化乘加指令的形式。

对于本文的 SVD 场景而言，这种适合硬件向量化执行的形式并不是通用矩阵乘法的完整外层调度结构，而是更基本的**向量点积**。其原因在于：无论是第一阶段的 $tmp = xV$，还是第二阶段的 $y = tmpU$，从逐输出元素的角度看，本质上都是一个输入向量与某一行权重之间的内积。例如，第一阶段可写为

$$tmp_i = \sum_j x_j V_{j,i}$$

第二阶段可写为

$$y_i = \sum_j tm\,p_j U_{j,i}$$

可以看到，这两步运算天然都能表述为 “两个向量之间的点积”。而点积恰好是最适合映射到底层向量化乘加指令的一类基本操作：其数据访问模式规则，乘加过程连续，易于在向量寄存器中并行展开，因此能够更充分地发挥底层 SIMD 或整型矩阵乘扩展的能力。

也正因为如此，本文并没有在自定义 SVD 算子中采用传统三层循环逐项完成乘加，而是进一步将两阶段计算都改写为向量点积序列。这样做的意义在于：一方面，它在算法组织层面避免了两次通用矩阵乘法调度的重复固定开销；另一方面，它又在底层实现层面为向量化乘加指令提供了最自然的映射对象。换言之，**向量点积并不是本文额外引入的一种数学近似，而是为了让专用指令能力真正发挥作用而选择的执行组织形式。**

在此基础上，本文复用了 llama.cpp 已有的高效点积实现。这样做具有两方面优势：其一，llama.cpp 的点积内核已经针对目标平台做过较成熟的底层优化，能够直接利用对应平台上的向量化乘加能力；其二，直接复用现有高效点积实现，可以使本

文的 SVD 专用算子与原有运行时保持一致的底层执行风格，避免重新实现底层内核所带来的额外复杂性与潜在错误风险。

因此，本文在这一部分的总体实现思路可以概括为：**为了最大化硬件执行并行度，首先利用底层向量化乘加能力作为性能基础；为了使这类指令能力能够直接作用于 SVD 两阶段计算，进一步将专用算子内部组织为向量点积形式；在具体实现上，则直接复用 llama.cpp 现有的高效点积内核完成底层执行。** 由此，本文的 SVD 专用算子既避免了两次通用矩阵乘法所带来的重复调度开销，又能够尽可能发挥底层硬件的并行执行能力，从而更适合单 token decode 热路径这一高频、小规模场景。

6.2.5 基于高效张量运算库的卷积算子设计与实现

6.2.6 面向 SVD 场景的高效卷积算子融合

6.3 基于 Llama.cpp 的协同推理计算图设计与实现

7 实验与结果分析

7.1 实验环境与设置

为验证本文所提出的基于 SVD 的弹性补偿计算方法、无原始数据场景下的精度评估算法以及基于负载感知的边端协同联合实时调度策略，本文在两类边侧设备与一类端侧设备上搭建实验平台，并围绕不同模型类型、不同协同方向以及不同通信链路条件开展实验。

7.1.1 实验平台

本文采用两台个人电脑作为边侧设备，分别代表不同处理器架构与性能层级下的边侧计算环境；采用一台手机作为端侧设备，用于执行端侧补偿计算或承担后层卸载任务。具体硬件配置如下：

1. **边侧设备 A**：Intel Xeon Gold 5218R 服务器平台。该设备为双路处理器配置，共 2 个 CPU 插槽，每颗处理器包含 20 个物理核心、40 个逻辑线程，总计 40 个物理核心与 80 个逻辑线程。该平台具有较强的多核并行处理能力，适合承担 Transformer 类模型的主要推理计算任务。
2. **边侧设备 B**：AMD Ryzen 7 6800H 平台。该设备代表个人电脑侧较常见的高性能移动处理器平台，用于验证在消费级边侧设备上本文方法的适用性。
3. **端侧设备**：一加 15 手机，采用 SM8850 芯片 CPU。该设备作为端侧协同计算节点，用于执行边侧卸载后的后层任务，或在传统 CNN 模型场景中作为主要运行

设备。

上述三类设备共同构成本文实验中的边端协同推理平台。由于两台边侧设备在处理器架构、核心规模和可用并行能力上存在明显差异，因此可以用于验证本文方法在不同边侧算力等级下的适应性；而手机端则用于刻画端侧设备在实际协同推理中的计算能力与通信约束。

此外，为减少操作系统后台任务及无关用户进程对实验结果的干扰，本文在边侧设备实验中采用 cgroup（control group）机制 对参与实验的 CPU 核心进行隔离与约束。具体而言，实验过程将推理任务绑定到预先划定的核心集合中运行，并通过 cgroup 限制其他无关任务占用该核心集合，从而降低系统调度抖动和资源竞争对测量结果的影响。通过这一方式，可以在较稳定的资源分配条件下评估不同裁剪率、调度策略和边端协同方案对推理时延、吞吐率及精度的实际影响，提高实验结果的可重复性与可信度。

7.1.2网络拓扑与协同链路

为考察不同通信方式对边端协同推理性能的影响，本文搭建了两种实验拓扑：

1. **Intel 边侧设备 — 手机端：WiFi 无线传输链路**
   该拓扑中，Intel Xeon 平台作为边侧主执行设备，手机端通过 WiFi 与其建立无线通信连接。该链路主要用于验证在无线网络条件下，Transformer 类模型从边侧向端侧卸载后层任务时的时延与调度特性。
2. **AMD 边侧设备 — 手机端：ADB 有线直连链路**
   该拓扑中，AMD Ryzen 7 6800H 平台与手机通过 ADB 有线方式直接连接。该链路主要用于构造相对稳定、低波动的边端通信环境，用于分析在有线直连条件下联合调度策略的行为特征，并与无线链路结果进行对比。

上述两类链路分别对应无线协同与有线协同两种典型边端通信场景。前者更贴近真实移动协同环境，能够反映网络波动对边端推理的影响；后者则有助于减少通信不确定性，从而更清晰地观察算法本身的调度效果。

7.1.3实验模型

本文选取两类代表性模型作为实验对象，分别对应 Transformer 架构模型与传统卷积神经网络模型。

1. **Transformer 类模型**
   - Qwen2.5-1.5B
   - Llama3.2-1B
2. **CNN 类模型**
   - ResNet50

选择上述模型的原因在于：一方面，Qwen2.5-1.5B 与 Llama3.2-1B 均属于当前具有代表性的轻量级 Transformer 架构模型，参数规模适合在边侧与端侧协同环境中进行推理实验，能够较好反映本文所研究的主贡献项 — 补偿项划分、层内裁剪与后层卸载策略在大语言模型场景下的适用性；另一方面，ResNet50 作为经典卷积神经网络模型，

层级结构清晰、卷积算子占比高，适合用于验证本文基于卷积矩阵化与 SVD 分解的统一建模方法在传统 CNN 场景中的有效性。
在实验中，Transformer 模型主要用于验证边 → 端协同路径，即边侧设备承担主要推理任务，在高负载场景下将模型后层卸载到手机端执行；ResNet50 则主要用于验证端 → 边协同路径，即手机端作为默认运行设备，在高负载场景下将模型后层卸载到边侧设备执行。

7.1.4设备角色与协同方向设置

根据模型类别与默认运行位置的不同，本文在实验中考虑两类协同方向：

1. **Transformer 类模型：边 → 端协同**
   对于 Qwen2.5-1.5B 与 Llama3.2-1B 两类 Transformer 模型，默认由边侧设备承担主要推理任务。当边侧设备负载较高且本地动态规划无可行解时，从模型后层开始向手机端逐步卸载计算任务，从而形成边 → 端协同推理路径。
2. **CNN 类模型：端 → 边协同**
   对于 ResNet50，默认由手机端承担主要推理任务。当手机端 CPU 占用较高时，首先在端侧本地通过卷积矩阵化与 SVD 分解，将关键卷积层划分为主贡献项与补偿项；若端侧本地仍无法满足时限要求，则从模型最后几层开始向边侧设备逐步卸载计算任务，从而形成端 → 边协同推理路径。

通过上述设置，本文实验不仅验证本文方法在单一方向协同推理中的有效性，也验证其在不同模型类别下支持双向协同的能力。

7.1.5量化设置

在边端协同实验中，手机端采用 **4bit 权重量化** 执行卸载后的模型层计算。该设置主要基于以下考虑：

1. 手机端存储与内存带宽资源相对受限，采用 4bit 权重量化有助于降低模型参数占用与权重访存开销；
2. 低比特量化可显著减小端侧执行压力，使手机端更适合承担边侧卸载后的后层任务；
3. 但与此同时，4bit 量化会引入额外量化误差，因此在实验中需要重点分析“卸载层数增加”与“量化精度退化”之间的关系。

基于此，本文在高负载协同实验中始终遵循“**在满足实时约束前提下尽可能少地将层卸载到手机端**”的原则，以避免由于端侧低比特量化导致整体精度损失过高。

7.1.6实验目标与评价标准

围绕本文提出的整体方法，实验主要考察以下几个方面：

1. **基于 SVD 的主贡献项 — 补偿项划分有效性**

验证不同裁剪率下主路径执行时间、补偿部分规模与模型精度变化之间的关系。

2. **无原始数据场景下精度评估方法的有效性**
   验证基于输入空间拟合生成的代理样本，能否有效估计不同协同策略下模型精度变化趋势。
3. **基于动态规划的本地实时调度有效性**
   验证在边侧设备或端侧设备本地负载变化条件下，动态规划方法能否合理分配裁剪率与超时预算，并尽可能维持主路径执行速度。
4. **高负载场景下边端协同联合调度有效性**
   验证当本地动态规划无解时，系统能否通过从后层开始逐步卸载任务恢复整体实时性，并找到最少的可行卸载层集合。

针对不同类型模型，本文采用如下评价指标。

（1）面向 Transformer 类模型的评价指标

对于 Qwen2.5-1.5B 和 Llama3.2-1B 两类大语言模型，本文主要采用以下指标：

- **吞吐率（token / s）**：表示单位时间内生成的 token 数量，用于衡量系统的生成效率。该指标能够直接反映不同裁剪策略、卸载策略以及协同调度策略对模型推理速度的影响。
- **首 token 延迟（TTFT, Time To First Token）**：表示从输入请求发起到生成第一个 token 所经历的时间，用于衡量系统交互响应速度。对于对话式与 Agent 类应用场景，该指标具有较强的实际意义。
- **单 token 解码时延**：表示生成阶段每个 token 的平均解码耗时，用于分析弹性融合与边端协同策略对逐步生成过程的影响。
- **困惑度(Perplexity, PPL)**：用于衡量语言模型在测试语料上的预测质量。PPL 越低，说明模型对文本分布的拟合能力越强。本文采用该指标评估不同裁剪率、不同卸载策略以及 4bit 量化执行对语言模型输出质量的影响。

（2）面向 CNN 模型的评价指标

对于 ResNet50 等传统卷积神经网络模型，本文主要采用以下指标：

- **分类准确率（Accuracy, Acc）**：表示模型在测试集上的分类正确比例，用于衡量卷积模型在不同裁剪与协同策略下的任务性能变化。
- **单样本推理时延**：表示单张输入图像完成一次前向推理所需的时间，用于刻画在手机端本地执行、端 → 边协同执行等不同条件下的实时性差异。

（3）通用系统级评价指标

除上述模型特定指标外，本文还关注以下系统级指标：

- **主路径执行时间**：用于衡量主贡献部分是否能够满足既定的目标请求时间约束。
- **总推理时间**：表示从输入请求发起到最终输出生成完成所经历的总时间，用于综合评估本地执行与协同执行方案的实时性。
- **是否满足实时约束**：用于判定当前调度策略是否满足设定的最大时延要求。
- **补偿任务完成率**：表示在总允许超时预算范围内，各层补偿分支成功完成的比例，用于反映弹性融合机制的有效性。
- **卸载层数**：表示在高负载场景下被卸载到另一侧设备执行的网络层数量。该指标用于衡量联合调度策略恢复可行性所需的最小卸载范围。
- **传输时延**：表示边端之间进行中间特征传输所需的时间，用于分析不同链路条件（WiFi 无线、ADB 有线）下协同推理的通信代价。

其中，token / s、TTFT、单 token 解码时延与 PPL 主要用于评估大语言模型的生成

效率与语言建模质量；Acc 主要用于评估 CNN 模型在分类任务上的精度保持能力；而主路径执行时间、总推理时间、补偿任务完成率、卸载层数和传输时延则用于衡量本文方法在系统级实时调度与边端协同执行中的整体表现。

7.1.7 实验组织说明

根据前文方法设计，本文后续实验将围绕三条主线展开：

第 3 章相关方法验证：验证卷积矩阵化、SVD 划分以及主贡献项 — 补偿项差异化执行机制的有效性；

第 4 章相关方法验证：验证无原始数据条件下精度评估算法对不同协同策略的估计能力；

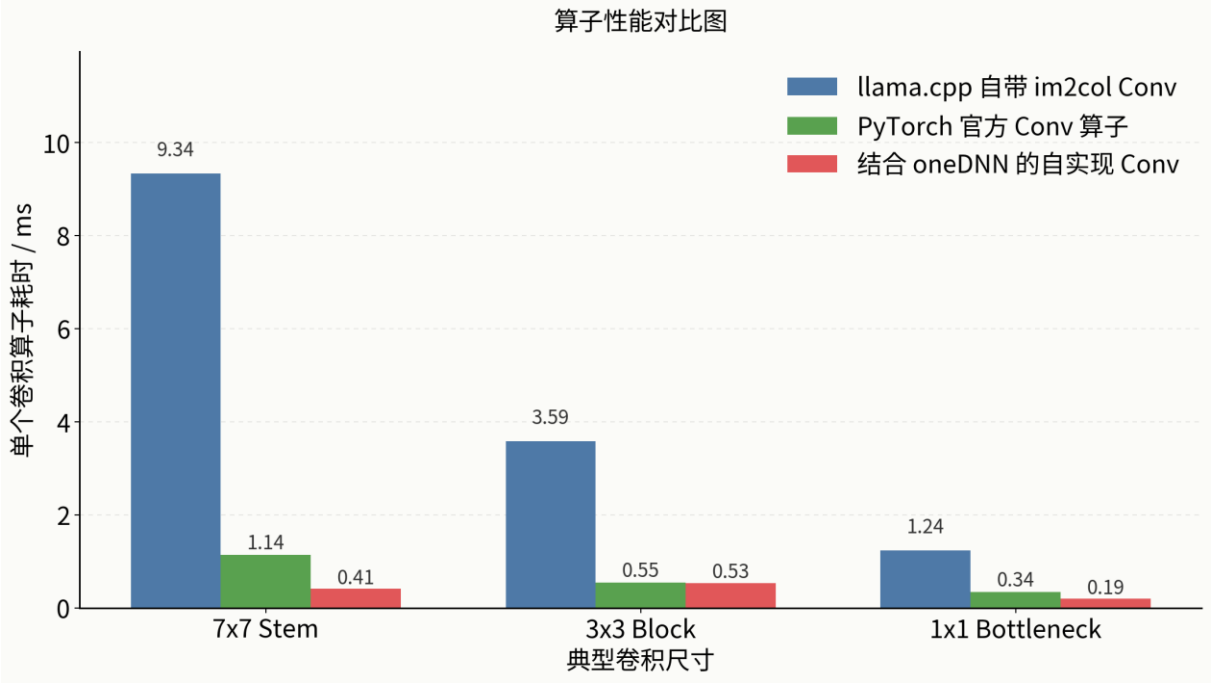
第 5 章相关方法验证：验证动态规划调度、本地弹性融合以及高负载场景下边端协同联合调度策略的有效性。

通过上述实验设置，本文构建了一个同时覆盖 Transformer 边 → 端协同与 CNN 端 → 边协同的统一实验平台，为后续验证所提出方法在不同模型类型、不同设备条件以及不同通信拓扑下的适用性提供了基础。

7.2 自研算子性能验证

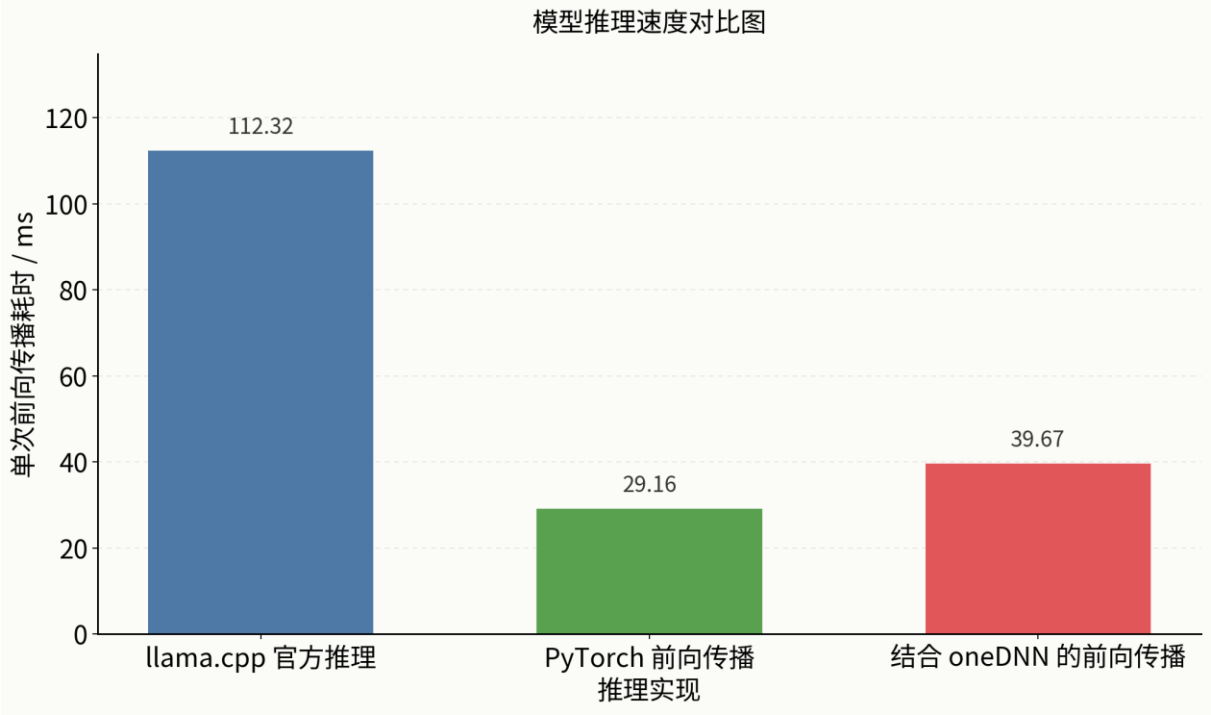
（1） 算子性能验证

为验证自研算子的性能与当前主流框架性能差距，做如下对比：本工作所实现的 Convolution，大矩阵乘法，与 Llama.cpp，pytorch 框架的算子性能对比。



（2） **模型推理速度验证**

为了进一步验证，本工作自研算子在实际推理过程中的性能，展示若干模型的推理速度。



再对比一下，使用 llama.cpp 提供的矩阵展开的算子实现的。

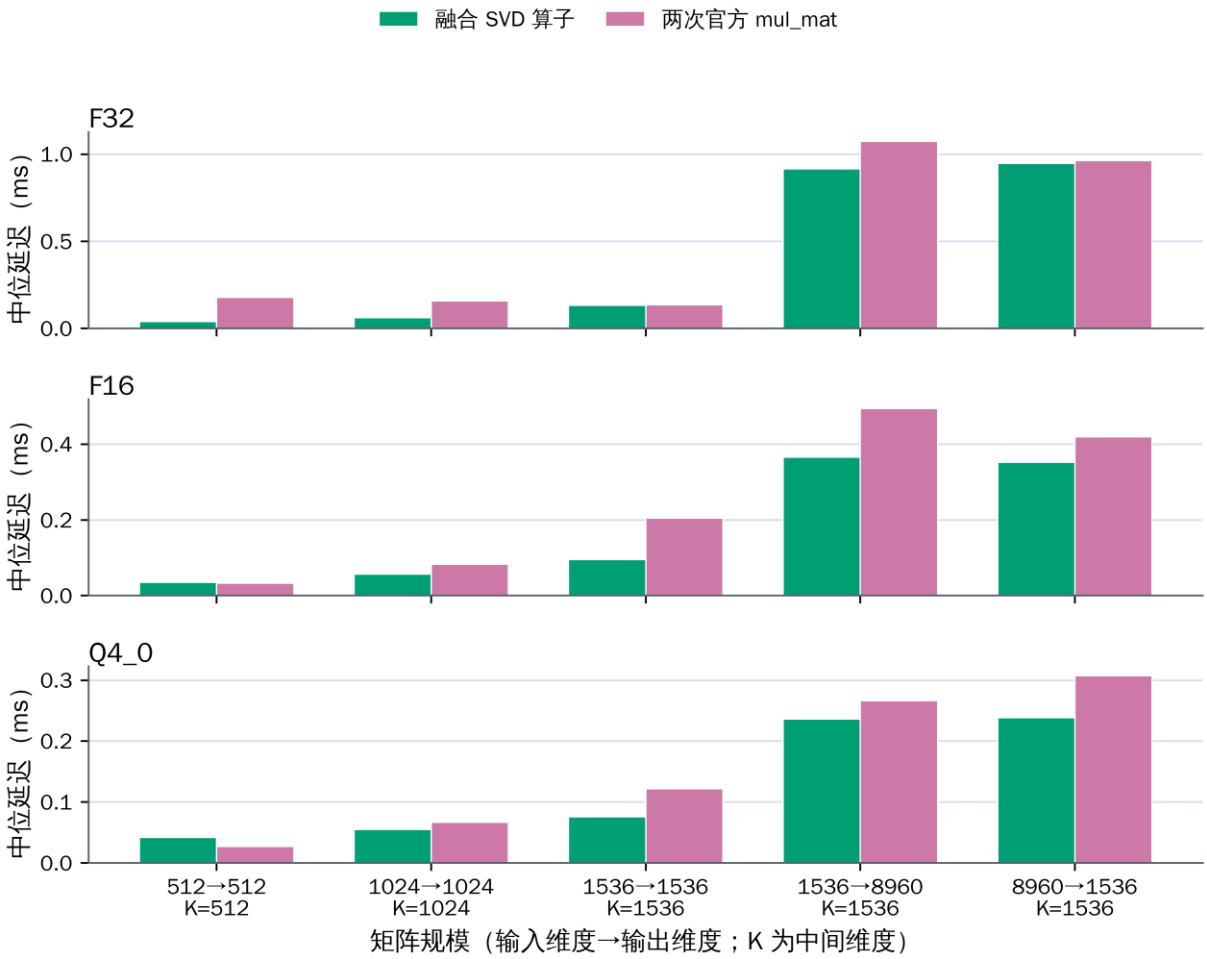
7.3 SVD 算子融合策略验证

大矩阵乘法与卷积算子融合策略性能验证。

（比较 llama.cpp 建议的两次算子调用；使用 im2col 算子；pytorch，以及自研融合算子的性能）

为了验证本文所提出的 SVD 算子优化方法对推理速度的提升效果，以及所设计的基于 Expansion-Fold 操作的内存访存优化机制在卷积场景下所带来的性能收益，本文围绕单设备执行场景开展如下两组实验。第一组实验用于考察在引入 SVD 分解并对尾部低重要度成分进行裁剪后，模型推理速度是否能够得到实际提升；第二组实验用于考察在传统卷积神经网络场景下，引入基于 Expansion-Fold 操作的卷积矩阵化访存优化后，相较于未引入该优化时能够获得多大的性能改善。通过上述实验，可以分别验证本文在 Transformer 类模型和 CNN 类模型场景下所提出算子级优化方法的有效性。

（1） **矩阵乘法算子融合加速验证**



（2） 卷积矩阵化的访存优化加速验证

该部分实验主要验证：对于传统 CNN 模型，在未引入基于 Expansion－Fold 操作的卷积矩阵化访存优化和引入该优化两种条件下，算子执行性能存在多大差异。实验将重点比较两种实现方式在推理时延和吞吐率上的表现，以分析该访存优化策略在改善卷积计算数据访问模式、提升算子执行效率方面的实际效果。

7.4 基于 SVD 分解的推理加速验证

该部分实验主要验证：在单一设备上，对经 SVD 分解后的算子丢弃尾部低重要度成分后，是否能够在保证主贡献路径计算的基础上带来实际推理加速。实验将比较不同裁剪率下的推理时延、吞吐率以及对应精度变化，用于分析基于 SVD 裁剪的算子优化在实际运行中的速度收益。

7.5 无原始数据场景下精度评估算法验证

对比不同裁减率下，使用生成数据对原始模型裁减时，原始模型的精度损失（百分比）和使用真实数据集中同样批量大小的数据，原始模型的精度损失（百分比）

7.6 基于动态规划的实时调度策略验证

设置不同的核心占用情况，展示最终经过调度策略调度后，推理速度的增加情况

7.7 高负载场景下的边端协同联合调度实验

设置不同的负载，不同网络拓扑情况，展示最终允许卸载到边缘设备后的，推理速度增加情况。
（1）

7.8 消融实验

（1） 不使用自己的构造小批量数据，而是使用随机数据作为输入，对比对最终协同推理决策后，精度损失的影响
（2） 使用自己构造的小批量数据，但不通过动态规划确定每一层的裁剪量，而是直接随机化的确定每一层裁剪量，再按照裁剪量估计精度损失，然后分配每层的超时量。
（3） 使用自己构造的小批量测试数据，通过动态规划确定每一层裁剪量，但平均分配超时量
（4） 使用随机小批量测试数据，使用随机化确认每一层裁剪量，平均分配超时量。

8 源码与开源贡献

（1） 提交 CNN 相关的高效算子代码合并请求到 llama.cpp 主分支，促进 llama.cpp 开源生态发展
（2） 本工作源码完全公开。

9 结论【1 级标题，三号黑体字】

［鼠标左键单击选择该段落，输入替换之。内容为小四号宋或楷体字。］论文的结论是最终的、总体的结论，不是正文中各段的小结的简单重复。结论应该准确、完整、明确、精练。如果不可能导出应有的结论，也可以没有结论而进行必要的讨论。可以在结论或讨论中提出建议、研究设想、仪器设备改进意见以及尚待解决的问题等。

参考文献

[内容为五号宋体。] 参考文献是文中引用的有具体文字来源的文献集合。按照 GB 7714《文后参考文献著录规则》的规定执行。

参考文献以文献在整个论文中出现的次序用[1]、[2]、[3]……形式统一排序、依次列出。

参考文献的表示格式为:

著作:

[序号] 作者.译者.书名.版本.出版地.出版社.出版时间.引用部分起止页

期刊:

[序号] 作者.译者.文章题目.期刊名.年份.卷号(期数).引用部分起止页

会议论文集:

[序号]作者.译者.文章名.文集名 .会址.开会年.出版地.出版者.出版时间.引用部分起止页

学位论文:

[序号]作者.题名[学位论文](英文用[Dissertation]).保存地点.保存单位.年份.引用部分起止页

专利:

[序号] 专利申请者.题名.国别.专利文献种类.专利号.发布日期.引用部分起止页

技术标准:

[序号] 起草责任者.标准代号.标准顺序号—发布年.标准名称.出版地.出版者.出版年份.引用部分起止页

致　　谢

［鼠标左键单击选择该段落，输入替换之。内容为小四号宋或楷体字。］放置在参考文献页后，对象包括：1）国家科学基金，资助研究工作的奖学金基金，合同单位，资助或支持的企业、组织或个人。2）协助完成研究工作和提供便利条件的组织或个人。3）在研究工作中提出建议和提供帮助的人。4）给予转载和引用权的资料、图片、文献、研究思想和设想的所有者。5）其他应感谢的组织和个人。